Rapport de Vulnérabilités Découvertes sur le site Bug Bounty Training FastFoodHackings

Date de Soumission : 05/10/2024

Soumis par :Aboubaker sane

Email : aboubakrbata@gmail.com

Programme de Bug Bounty : FastFoodHackings

Résumé **:**

Ce rapport présente les détails des vulnérabilités découvertes lors de l'évaluation de sécurité du site Web Bug Bounty Training FastFoodHackings. Les vulnérabilités identifiées incluent :

1. **Clickjacking**
2. **Cross-Origin Resource Sharing (CORS)**
3. **Broken Access Control**
4. **IDOR (Insecure Direct Object Reference)**
5. **HTML injection**
6. **Reflected XSS Vulnerability**
7. **une vulnérabilité de redirection ouverte**
8. **Fuite d'Identifiants**
9. **CVE-2020-12440**

Figure 1: FastFoodHackings

Table des Matières

Table des matières

Table des Matières 2
Table des figure 3
Introduction 4
Reconnaissance **5**
vulnerability assessment and Exploitation : **7**
Clickjacking : 7
Cross-Origin Resource Sharing (CORS) 11
vulnérabilité de redirection ouverte 13
Reflected XSS vulnerability. 14
Fuite d'Identifiants 15
IDOR: 18
HTML injection : 19
Revenons à la page d'accueil et recherchons quelque chose d'intéressant à tester. Mon intuition suggère que nous devrions examiner le formulaire de connexion. Nous pourrions découvrir une autre vulnérabilité 19
Broken Access Control : 21
CVE-2020-12440 : 27
Documentation : **28**
Conclusion **31**

Table des figure

FIGURE 1: FASTFOODHACKINGS ---------- 1
FIGURE 2:RESULTA DE SCAN NMAP ---------- 5
FIGURE 3:OSINT FRAMEWORK ---------- 5
FIGURE 4: AUTORECONX ---------- 6
FIGURE 5:NIKTO ---------- 6
FIGURE 6:BEEF ---------- 7
FIGURE 7:BEEF PAGE LOGIN ---------- 8
FIGURE 8:CLICKJACKING ---------- 8
FIGURE 9: IFRAME CHANGER ---------- 9
FIGURE 10:EXPLOITATION INTERFACE DE USER ---------- 9
FIGURE 11: L'ATTAQUE REUSSI ---------- 10
FIGURE 12:CODE IFRAME ---------- 11
FIGURE 13:CORS ---------- 11
FIGURE 14:ROBOTS.TXT ---------- 12
FIGURE 15 :VERSION DU SERVEUR ---------- 13
FIGURE 16 :TEST /GO.PHP ET ADMIN.PHP ---------- 13
FIGURE 17:AUTORECONX ---------- 14
FIGURE 18:XSS ---------- 15
FIGURE 19: INTERCEPTER LA REQUETE ---------- 16
FIGURE 20: REQUETE ---------- 16
FIGURE 21:CREDENTIAS ---------- 17
FIGURE 22:LOGIN PAGE DE ZSENO ---------- 17
FIGURE 23:REQUETE ID=PARAMETRE ---------- 18
FIGURE 24:IDOR ---------- 18
FIGURE 25:INSECURE DIRECT OBJECT REFERENCE ---------- 19
FIGURE 26:LOGIN TEST ---------- 19
FIGURE 27:L'INJECTION HTML ---------- 20
FIGURE 28:XSS TROUVE ---------- 21
FIGURE 29:TERME BETA ---------- 22
FIGURE 30:L'EDITEUR DE COOKIES ---------- 23
FIGURE 31:BONTTON : FOODHACKING ---------- 23
FIGURE 32:PAGE SENSIBLE ---------- 24
FIGURE 33: TROIS INPUT ---------- 24
FIGURE 34: UN ID DANS LE REPONSE ---------- 25
FIGURE 35:VERIFIONS LES AUTRE REQUETS ---------- 25
FIGURE 36:ENCODER BASE64 ---------- 25
FIGURE 37:CONTROLE D'ACCES BRISE ---------- 26
FIGURE 38: CHANGER LE NUMERO BASE64 ---------- 26
FIGURE 39:CVE-2020-12440 ---------- 27

Introduction

Ce rapport présente les vulnérabilités de sécurité découvertes sur le site web FastFoodHackings lors des tests réalisés dans le cadre du programme de Bug Bounty. Chaque vulnérabilité est décrite en détail, avec des étapes pour la reproduire, une analyse de l'impact potentiel sur la sécurité, ainsi que des recommandations de correction pour aider les développeurs à sécuriser la plateforme. Ce document vise à fournir un aperçu clair des risques identifiés et à proposer des solutions concrètes pour atténuer ces failles.

Méthodologie

La méthodologie employée pour identifier les vulnérabilités dans FastFoodHackings comprend les étapes suivantes :

Reconnaissance :

La phase de reconnaissance consiste à recueillir des informations sur l'architecture et les fonctionnalités du site web, en analysant des fichiers comme robots.txt et d'autres points d'accès. Des outils de scan et de recherche, tels que AutoReconX (https://github.com/Sane-Sec/AutoReconX), ont été utilisés pour automatiser et faciliter la collecte d'informations cruciales pendant cette phase.

vulnerability assessment et Exploitation :

Durant cette phase, des outils comme OWASP ZAP et Nikto ont été employés pour scanner le site et identifier les vulnérabilités. Nous avons ensuite utilisé Burp Suite pour tester des scénarios spécifiques d'attaque, tels que l'injection de payloads malveillants et la manipulation des paramètres pour détecter des failles.

Une fois les vulnérabilités identifiées, nous avons tenté d'exploiter les failles pour en valider l'impact réel. Chaque attaque a été soigneusement documentée avec des captures d'écran et des preuves d'accès pour montrer la gravité des vulnérabilités découvertes.

Documentation :

Toutes les vulnérabilités découvertes ont été enregistrées avec des détails complets concernant leur reproduction, leur impact, et les résultats obtenus, afin de fournir un rapport exhaustif aux développeurs pour qu'ils puissent rapidement corriger ces problèmes.

Reconnaissance

Interagir directement avec la cible pour obtenir des informations la cible :

Utiliser osint framework :

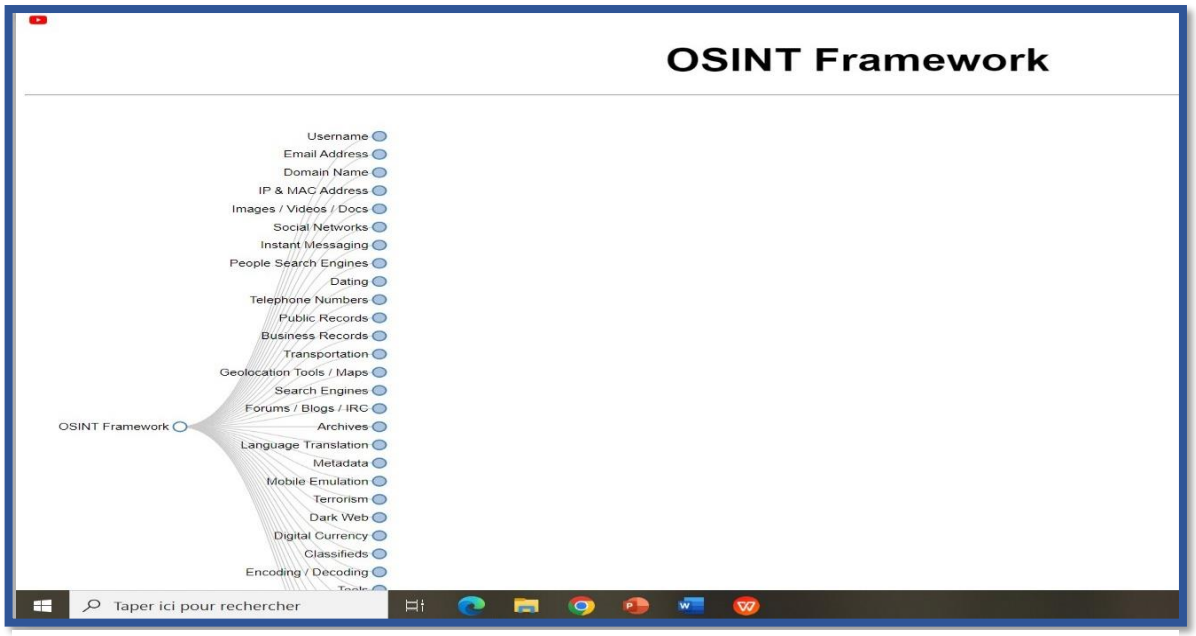

Figure 3:osint framework

```
Nmap scan report for bugbountytraining.com (134.209.18.185)
Host is up (0.13s latency).
Not shown: 97 filtered tcp ports (no-response)
PORT    STATE SERVICE
22/tcp  open  ssh
80/tcp  open  http
443/tcp open  https

Nmap done: 1 IP address (1 host up) scanned in 4.23 seconds
```

Figure 2:resulta de scan nmap

Utiliser AutoReconX:

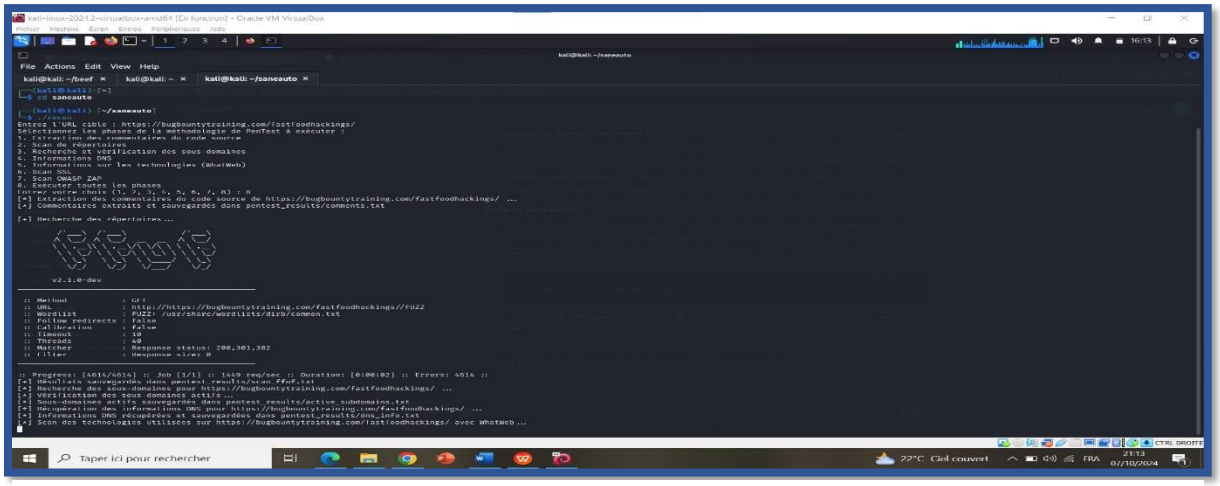

Figure 4: autoReconX

Utiliser nikto :

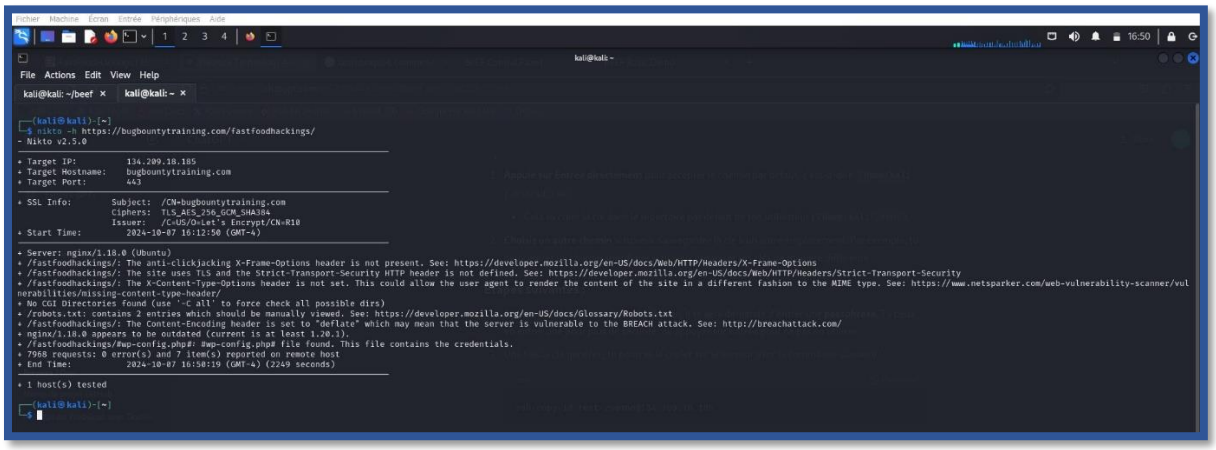

Figure 5:nikto

Voici les résultats du scan Nikto:

Le scan Nikto a révélé plusieurs points concernant le site `bugbountytraining.com`. Tout d'abord, le serveur utilise **nginx/1.18.0 (Ubuntu)** , qui est une version obsolète, la version actuelle étant au moins 1.20.1. De plus, plusieurs en-têtes de sécurité essentiels ne sont pas présents. L'en-tête **X-Frame-Options**, qui protège contre les attaques de type clickjacking, est absent. Le site utilise TLS, mais l'en-tête **Strict-Transport-Security (HSTS)** n'est pas défini, ce qui expose les utilisateurs à des attaques de type downgrade. L'en-**tête X-Content-Type-Options** est également manquant, ce qui pourrait permettre au navigateur de rendre le contenu du site de manière incorrecte. Le fichier **robots.txt** contient deux entrées qui nécessitent une révision manuelle. Enfin, l'en-tête **Content-Encoding** est réglé sur "deflate", ce qui pourrait rendre le serveur vulnérable à une attaque de type **BREACH.**

vulnerability assessment and Exploitation :

Clickjacking :

Le clickjacking est une attaque de sécurité dans laquelle un utilisateur est trompé pour cliquer sur un élément d'une page web sans s'en rendre compte, car cet élément est masqué ou superposé à une autre interface visible. L'attaquant utilise une technique pour rendre une interface invisible ou transparente, tout en la plaçant au-dessus d'un élément légitime, tel qu'un bouton ou un lien que l'utilisateur pense cliquer. Par exemple, un utilisateur peut croire qu'il clique sur un bouton "J'aime" sur un réseau social, alors qu'en réalité, il est en train d'autoriser une transaction malveillante ou d'accorder des privilèges à un attaquant. Pour se protéger contre ce type d'attaque, les développeurs web peuvent utiliser l'en-tête HTTP **X-Frame-Options**, qui empêche une page d'être chargée dans un cadre (iframe) en provenance de sites non autorisés, rendant ainsi plus difficile l'exécution de cette attaque.

Pour l'exploitation, nous exploiterons beef.

BeEF (The Browser Exploitation Framework) est un outil puissant qui permet d'exploiter les vulnérabilités des navigateurs web. Pour exploiter une attaque de clickjacking avec BeEF, voici les étapes générales que tu peux suivre *:*

Installation de BeEF

Si tu n'as pas encore installé BeEF, voici comment le faire sur Kali Linux

sudo apt-get install beef

Lance ensuite BeEF avec la commande suivante :

sudo beef

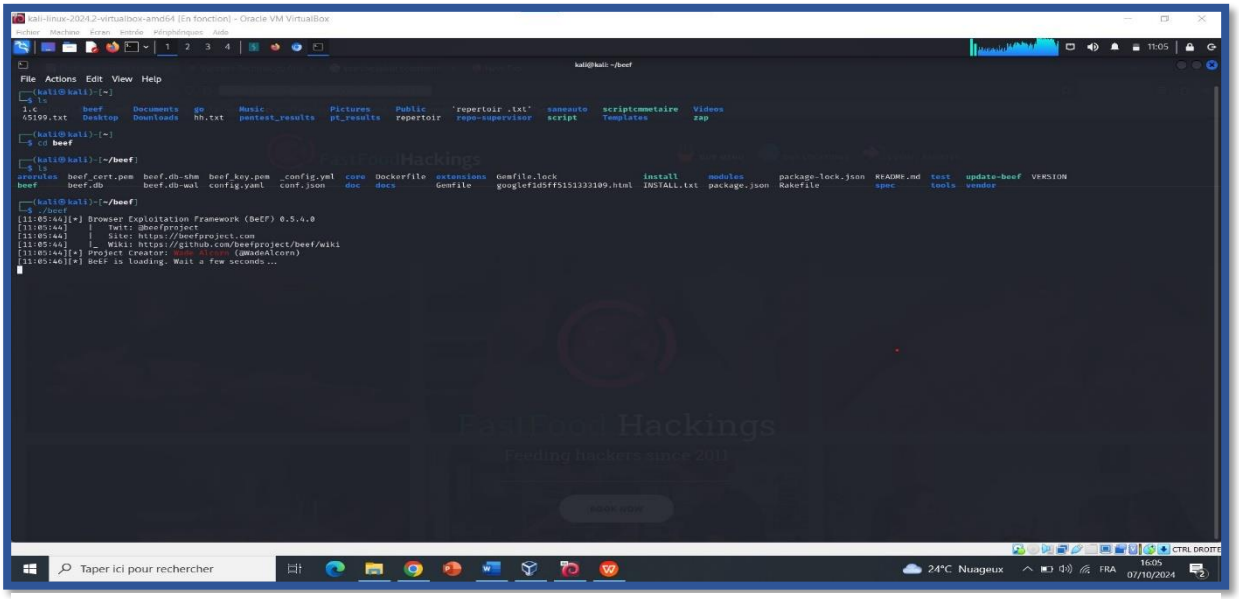

Figure 6:beef

Cela démarrera le serveur BeEF, généralement accessible via le navigateur à l'adresse http://localhost:3000/ui/panel.

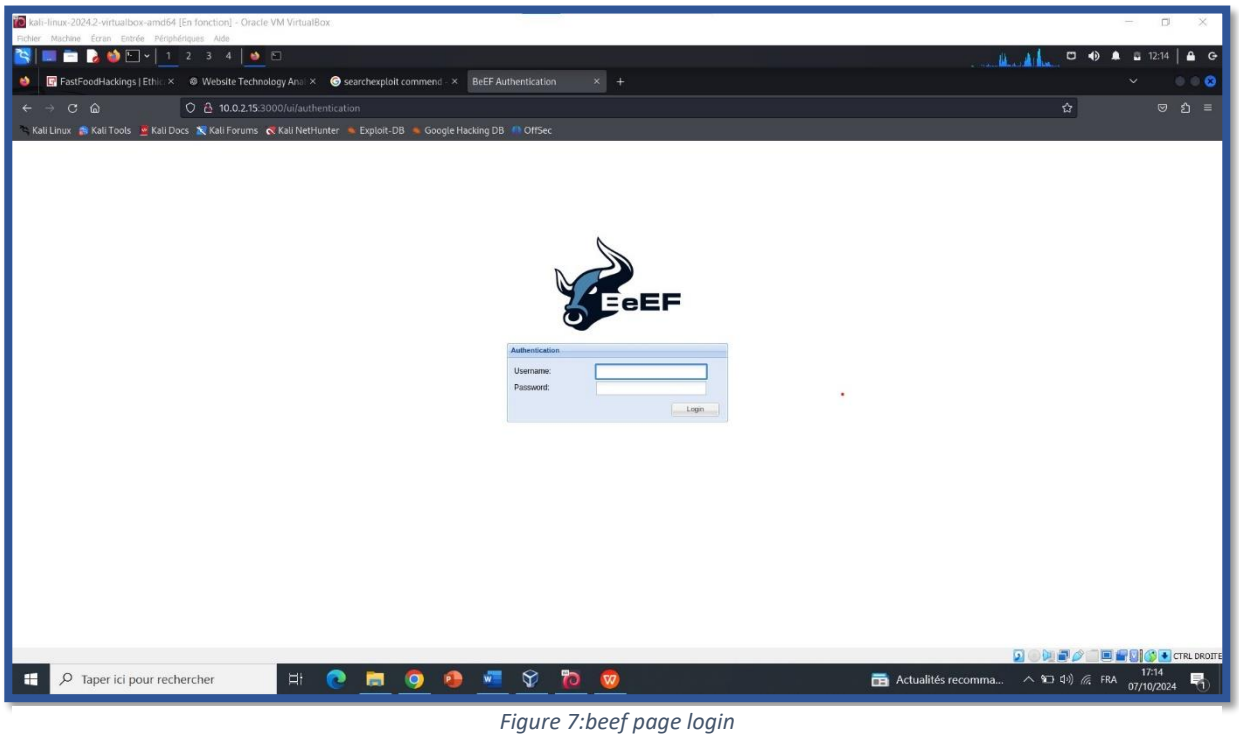

Figure 7:beef page login

Vous pouvez modifier votre mot de passe via le ficher configue-yamal.txt

Après m'être entré à beef Comme vous pouvez le voir dans le dossier d'ingénierie sociale nous choisissons Clickjacking:

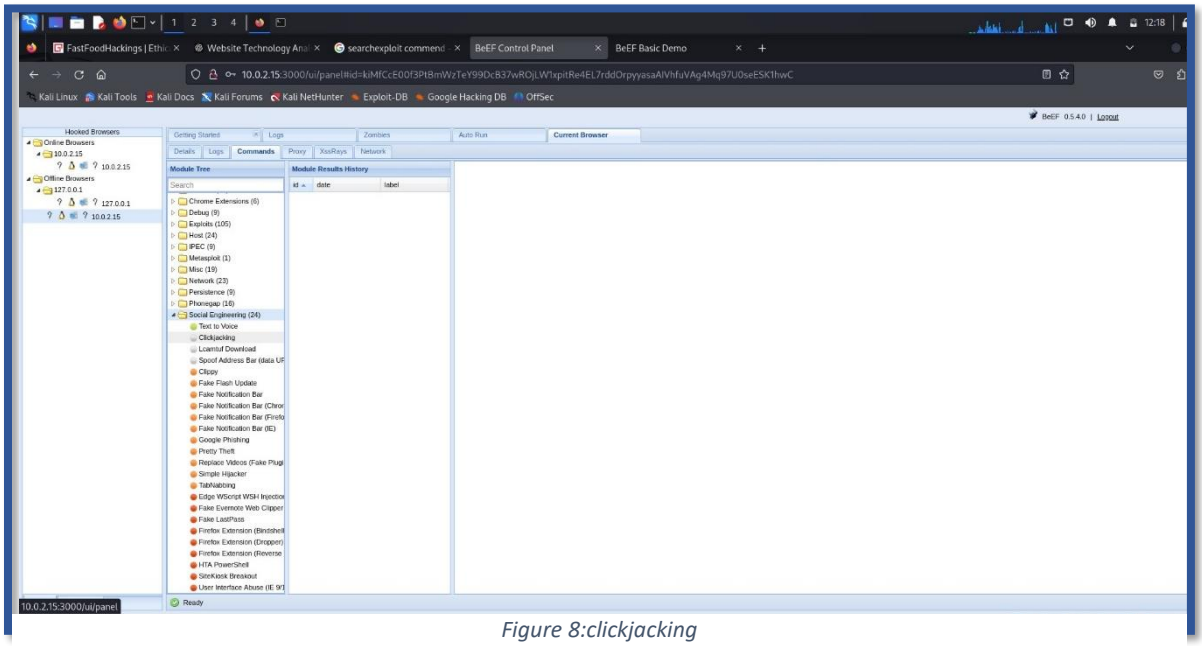

Figure 8:clickjacking

Nous allons changer iframe par url de la cible pour Cibler une victime.

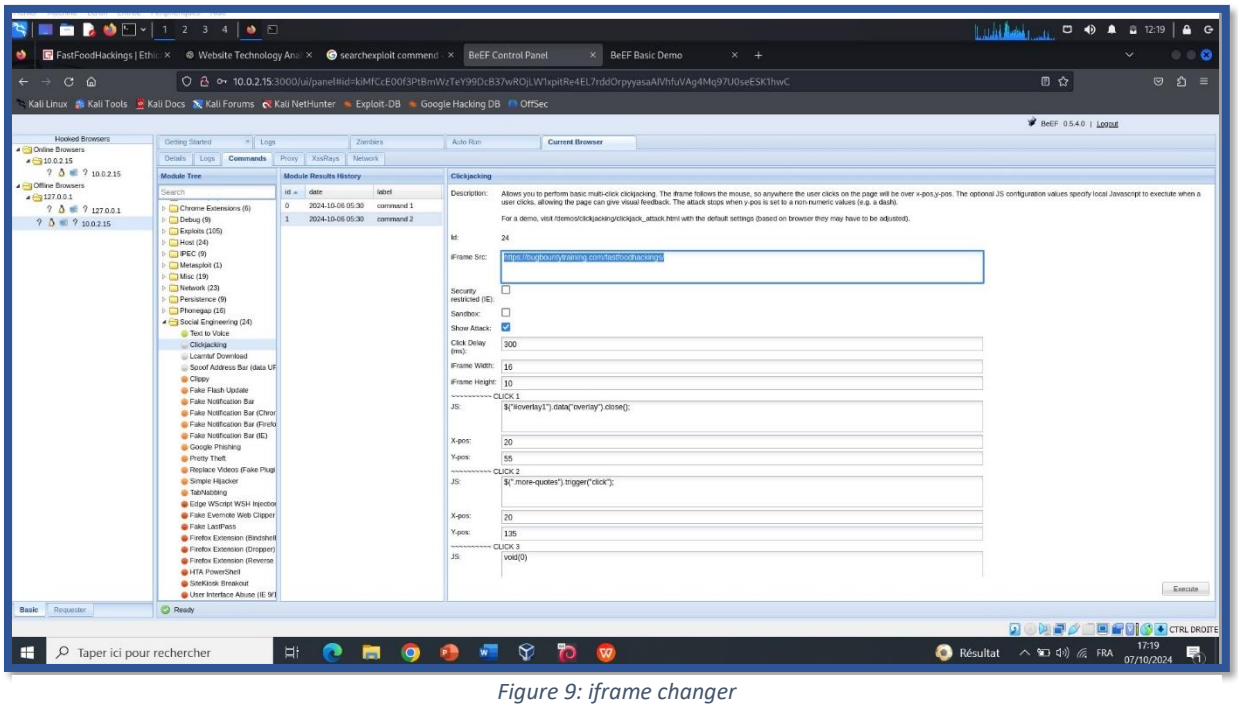

Figure 9: iframe changer

Il s'agit de l'interface de la victime : partout où la pression est appliquée, l'attaque réussira.

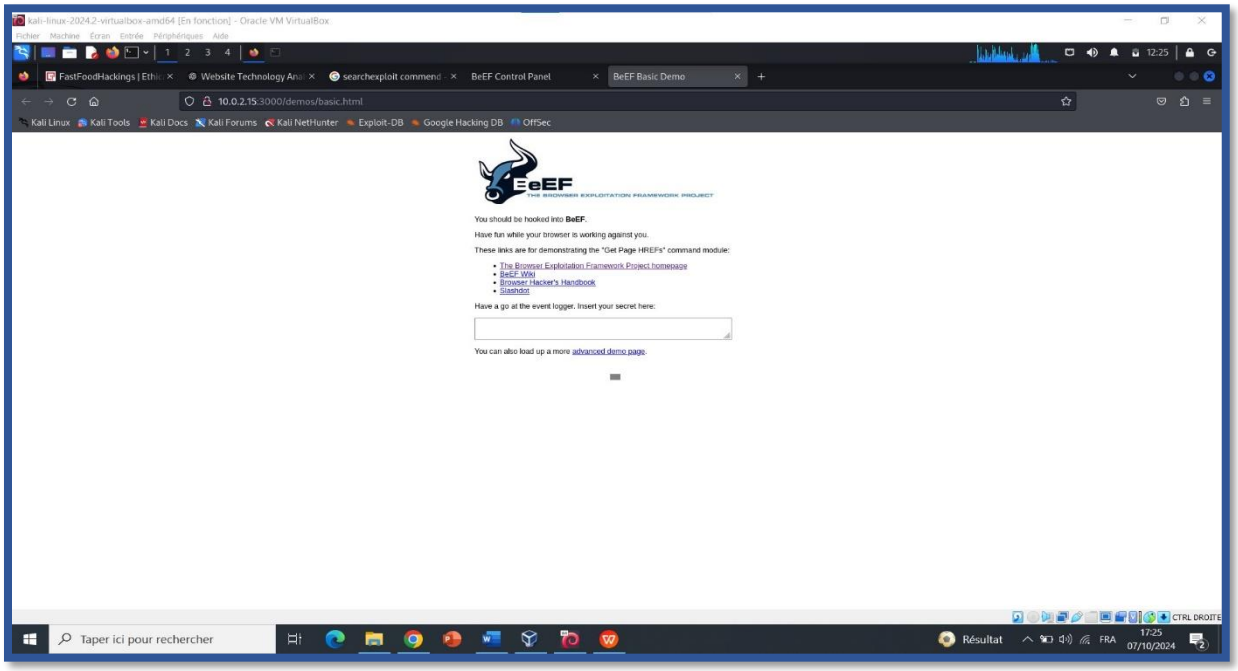

Figure 10:exploitation interface de user

Comme vous pouvez le constater, l'attaque a réussi.

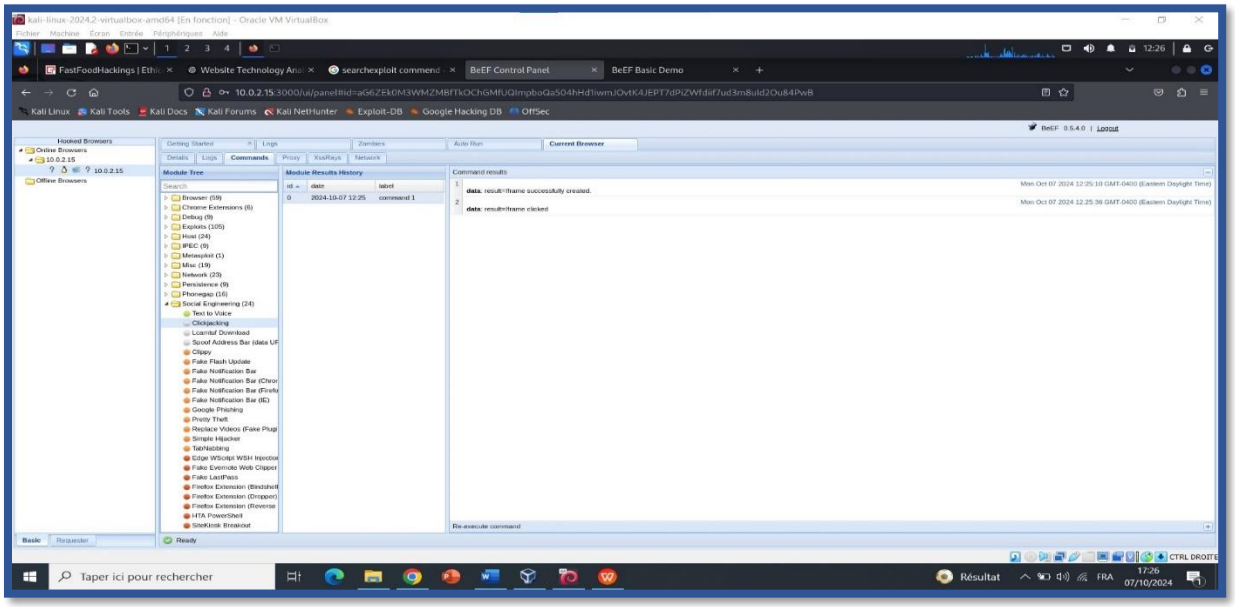

Figure 11: l'attaque réussi

Je comprends mieux comment cela se fait, j'ai écrit le code ci-dessous

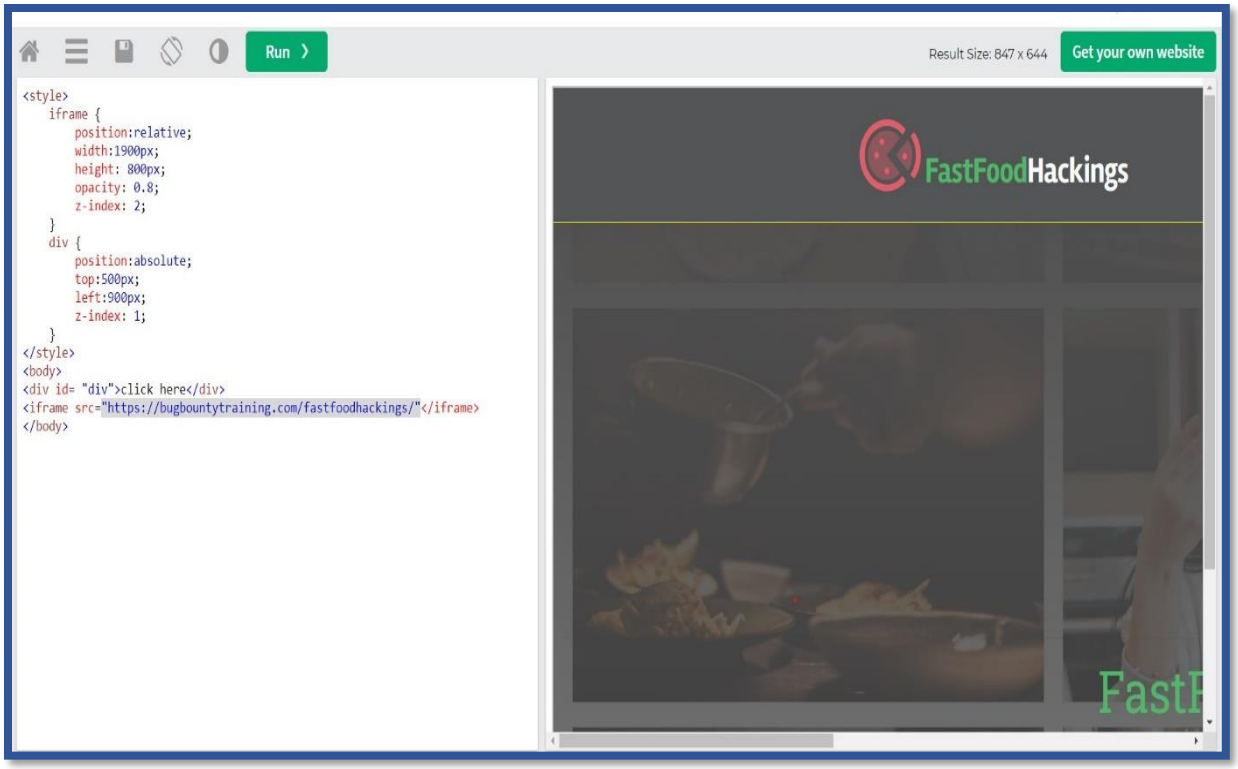

Figure 12:code iframe

Cross-Origin Resource Sharing (CORS)

Le Cross-Origin Resource Sharing est un mécanisme de sécurité qui permet aux serveurs web de contrôler les ressources qu'ils partagent avec des domaines externes, c'est-à-dire des origines différentes. Par défaut, pour des raisons de sécurité, les navigateurs bloquent les requêtes HTTP provenant d'une origine différente de celle du serveur, empêchant ainsi un site malveillant d'accéder à des données sensibles d'un autre site. CORS intervient en permettant aux serveurs d'autoriser explicitement certaines requêtes cross-origin via des en-têtes spécifiques, tels que

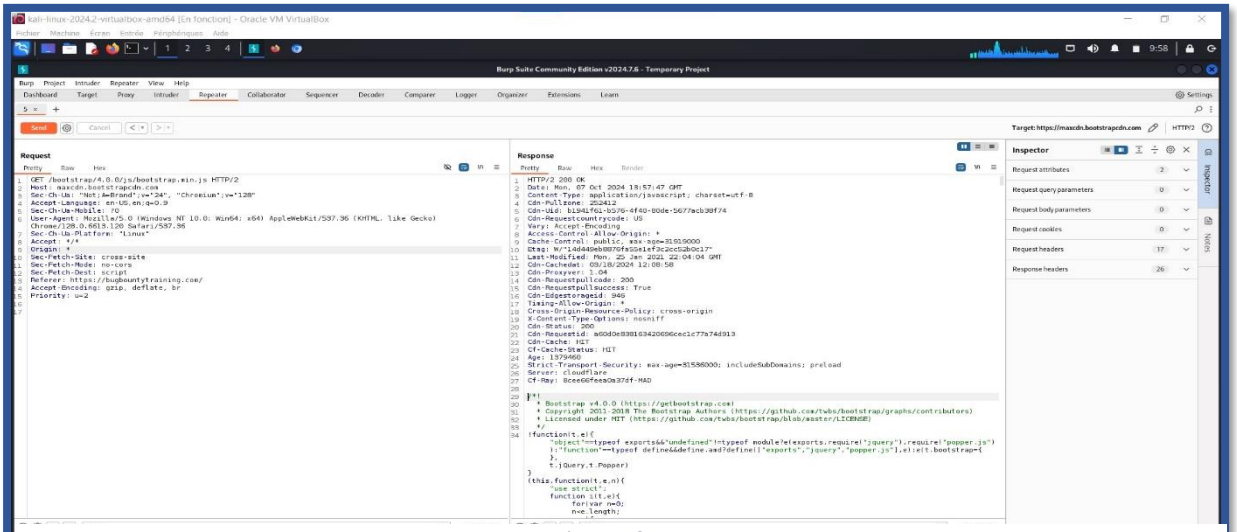

Figure 13:cors

Access-Control-Allow-Origin, qui indiquent les domaines autorisés à accéder aux ressources. Par exemple, un serveur peut répondre à une requête d'un autre domaine avec l'en-tête `Access-Control-Allow-Origin: `, autorisant toutes les origines à accéder à ses ressources. CORS protège donc contre certaines attaques telles que le vol de données entre sites, tout en offrant la flexibilité de partager des ressources de manière contrôlée entre différents domaines lorsque cela est nécessaire.

La capture montre une requête cross-origin pour un fichier JavaScript, et la réponse inclut les en-têtes qui permettent ce partage (grâce à Access-Control-Allow-Origin:), ce qui signifie que cette ressource est accessible à toutes les origines. Cela permet les requêtes **CORS** depuis n'importe quel domaine.

Lors du processus de collecte d'informations, nous avons découvert que les pages Web sont créées à l'aide du langage PHP. Cela peut être utile plus tard.

Notre prochaine étape consiste à vérifier le fichier robots.txt. Ce fichier peut révéler des informations importantes. Dans ce cas, deux dossiers nous sont présentés : admin et go. Explorons ces dossiers pour voir ce que nous trouvons.

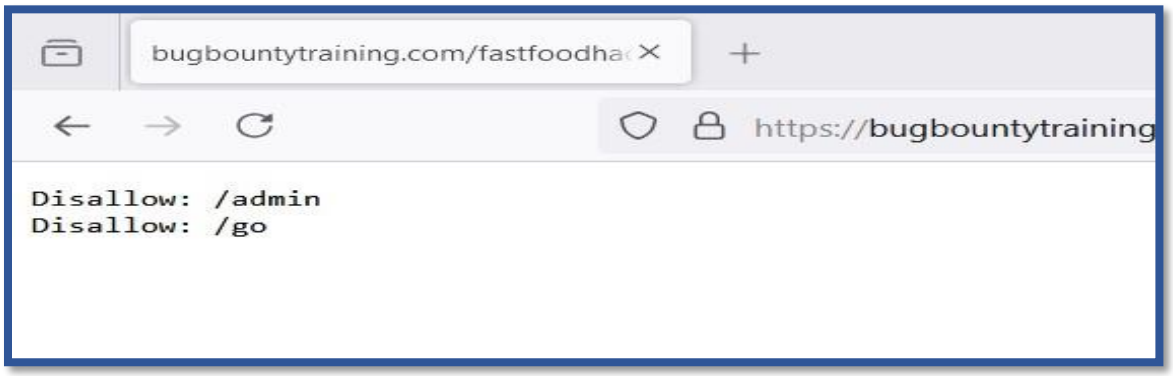

Figure 14:robots.txt

Après avoir exploré les répertoires adminet go, nous obtenons une 404 Not Founderreur. Cela signifie que ces répertoires n'existent pas ou ne sont pas accessibles.

Figure 15 :Version du serveur

Puisque nous savons que toutes les pages sont créées à l'aide de PHP, essayons d'ajouter .phpà la fin des noms de répertoires , d'abord, nous allons essayer admin.php. Cela fonctionne, mais cela nous redirige vers la page principale.

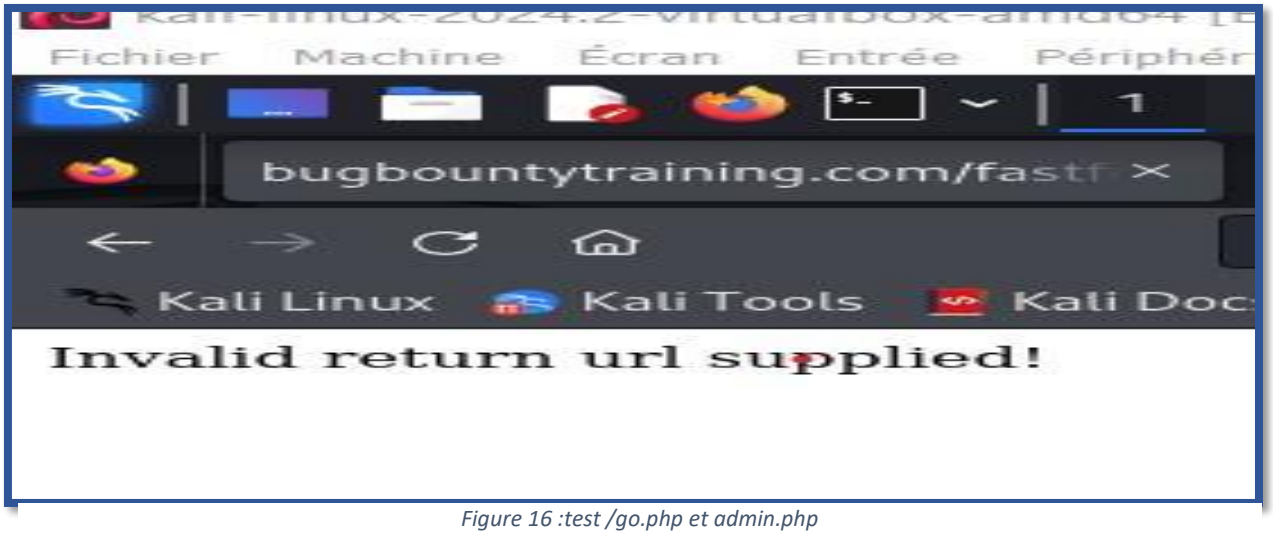

Figure 16 :test /go.php et admin.php

vulnérabilité de redirection ouverte

Après quelques recherches et tentatives, nous avons découvert une vulnérabilité de redirection ouverte . Nous pouvons exploiter cette vulnérabilité en utilisant la charge utile suivante :

```
https://www.bugbountytraining.com/fastfoodhackings/go.php?returnUrl=https://github.com/Sane-Sec/AutoReconX
```

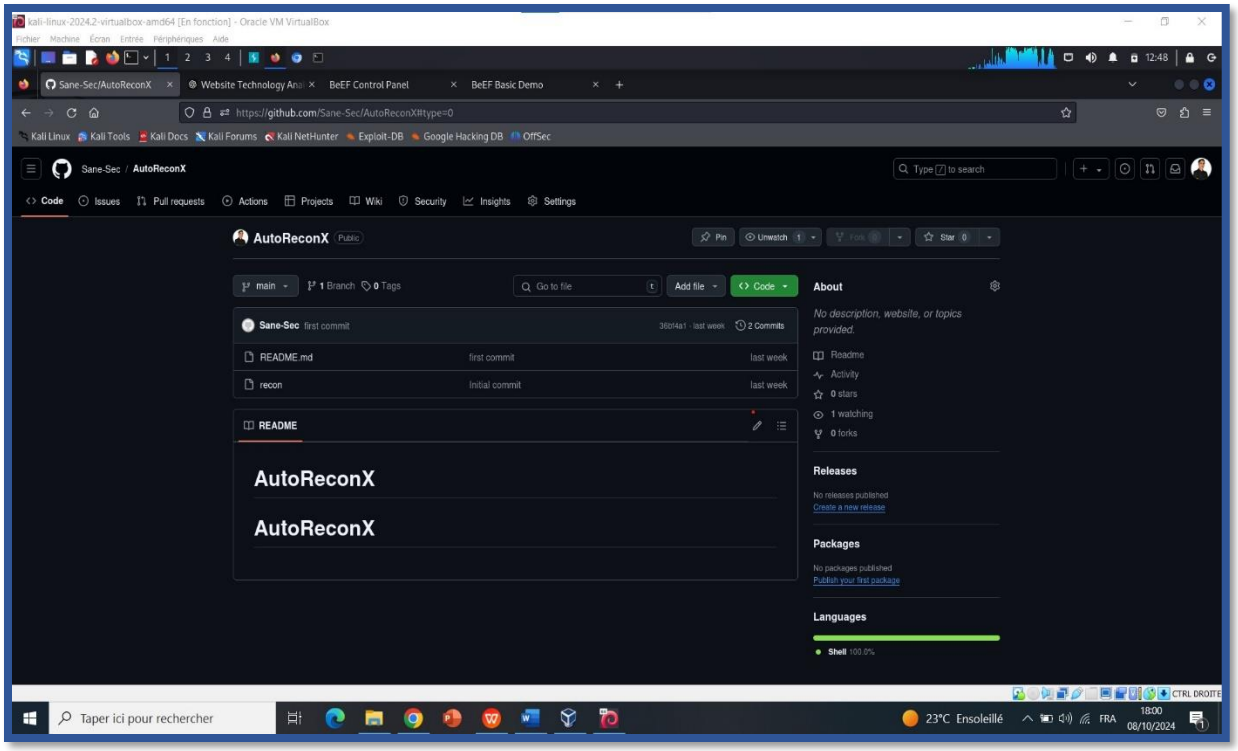

Figure 17:AutoreconX

Reflected XSS vulnerability.

Lors de la découverte d'une vulnérabilité de redirection ouverte, notre charge utile a redirigé avec succès vers github.com. Cependant, un élément intrigant dans l'URL a retenu mon attention : `/#type=0`. Soupçonnant une fonctionnalité liée à ce fragment, j'ai mené une enquête approfondie. Une recherche rapide a révélé que le site pourrait utiliser ce fragment pour permettre au JavaScript de modifier dynamiquement le contenu de la page sans rechargement complet. Curieux de tester cette hypothèse, j'ai intercepté la requête à l'aide de Burp Suite et remplacé le `0` par d'autres valeurs numériques afin d'explorer des comportements potentiellement intéressants. Cette approche s'est avérée fructueuse : en modifiant cette valeur, j'ai découvert des variations de contenu ou des fonctions cachées, dévoilant ainsi des informations sensibles ou des failles exploitables sur le site cible.

Cette modification a permis une découverte intéressante. En ajustant la valeur du paramètre `type`, la réponse du serveur a changé et contient désormais du code JavaScript. Ce comportement indique que le paramètre a un impact direct sur le contenu renvoyé par le serveur. Une telle observation est cruciale dans un contexte de tests de sécurité, car elle peut révéler des vulnérabilités potentielles telles que des failles de type Cross-Site Scripting (XSS). Cela met en évidence la nécessité d'analyser attentivement la manière dont les paramètres influencent la réponse du serveur, et d'explorer les différentes possibilités de manipulation pour identifier des failles exploitables.

https://www.bugbountytraining.com/fastfoodhackings/go.php?returnUrl=javascript:alert(1);//&type=1

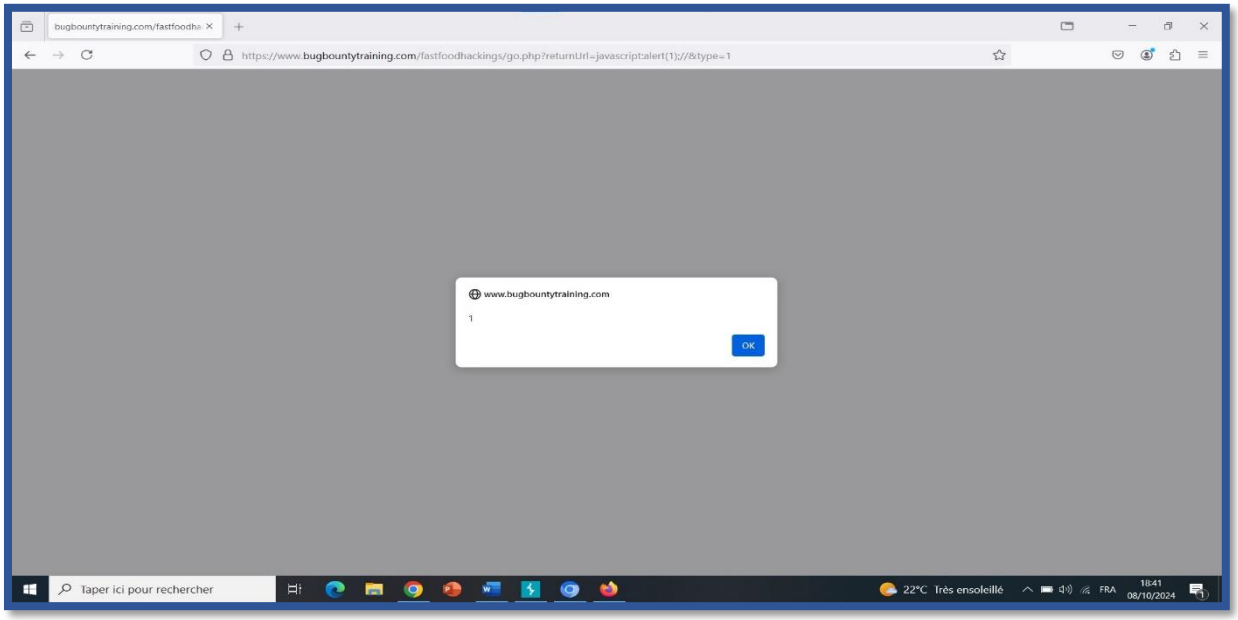

Figure 18:xss

Parfait ! Comme vous pouvez le voir, notre message d'alerte s'est exécuté sur la cible, indiquant que nous avons découvert une vulnérabilité XSS réfléchie .

Mon intuition suggère que nous devrions examiner le formulaire de connexion. Nous pourrions découvrir une autre vulnérabilité XSS, SQLI ou autre. Testons son comportement et voyons ce que nous pouvons trouver.

Fuite d'Identifiants

En examinant cette demande plus en détail et en recherchant des paramètres supplémentaires à tester, nous pourrions découvrir quelque chose d'intéressant.

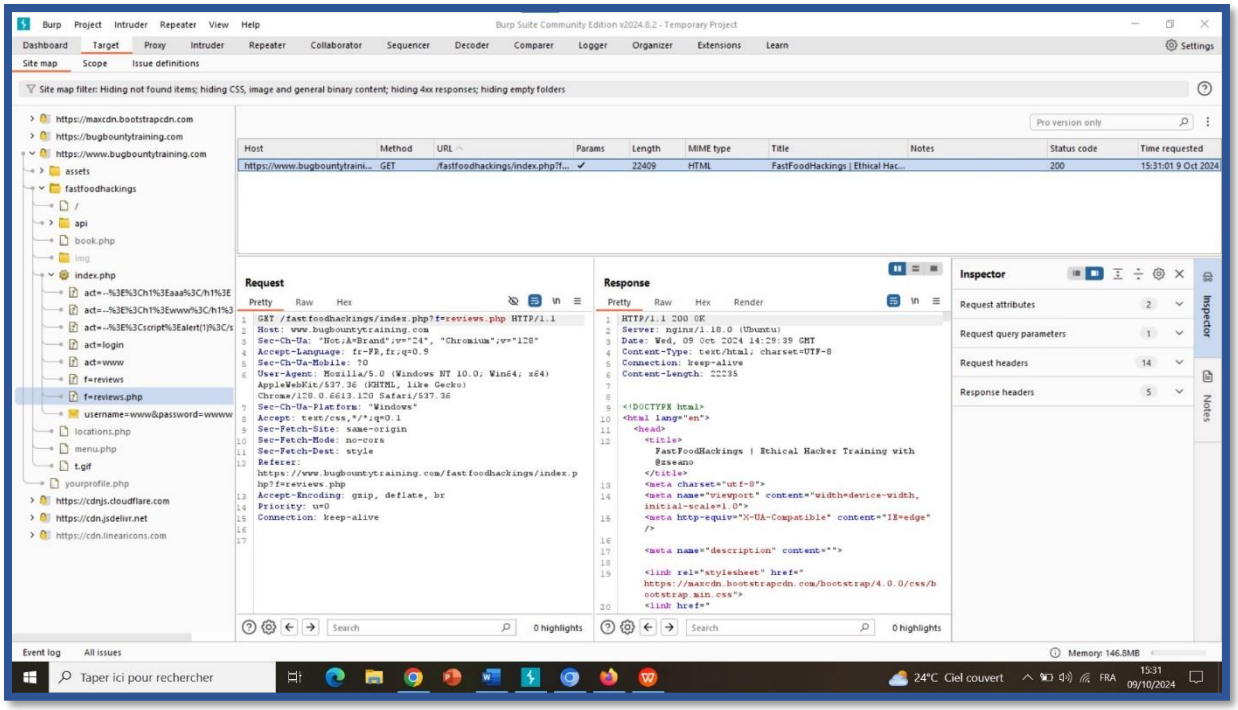

Figure 19: Intercepter la requête

Nous allons donc continuer à utiliser Burp Suite pour rechercher dans le code et les fichiers le paramètre f=

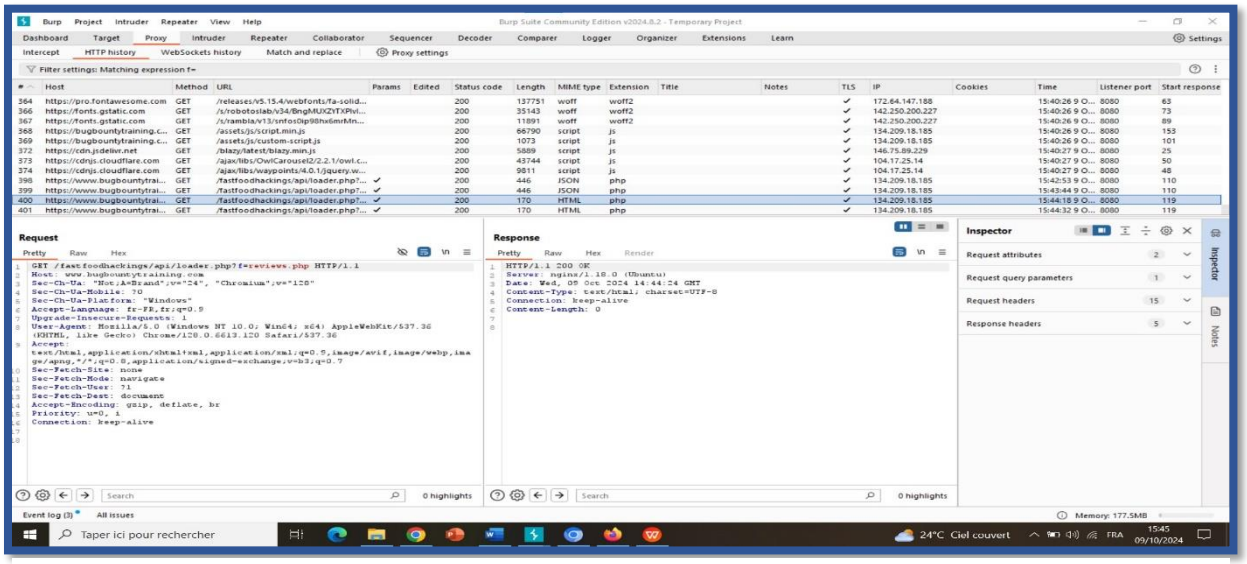

Figure 20: requête

Waouh ! C'est la plus belle découverte jusqu'à présent, nous avons enfin trouvé les identifiants ! Je suis presque sûr qu'il s'agit des identifiants d'administrateur.

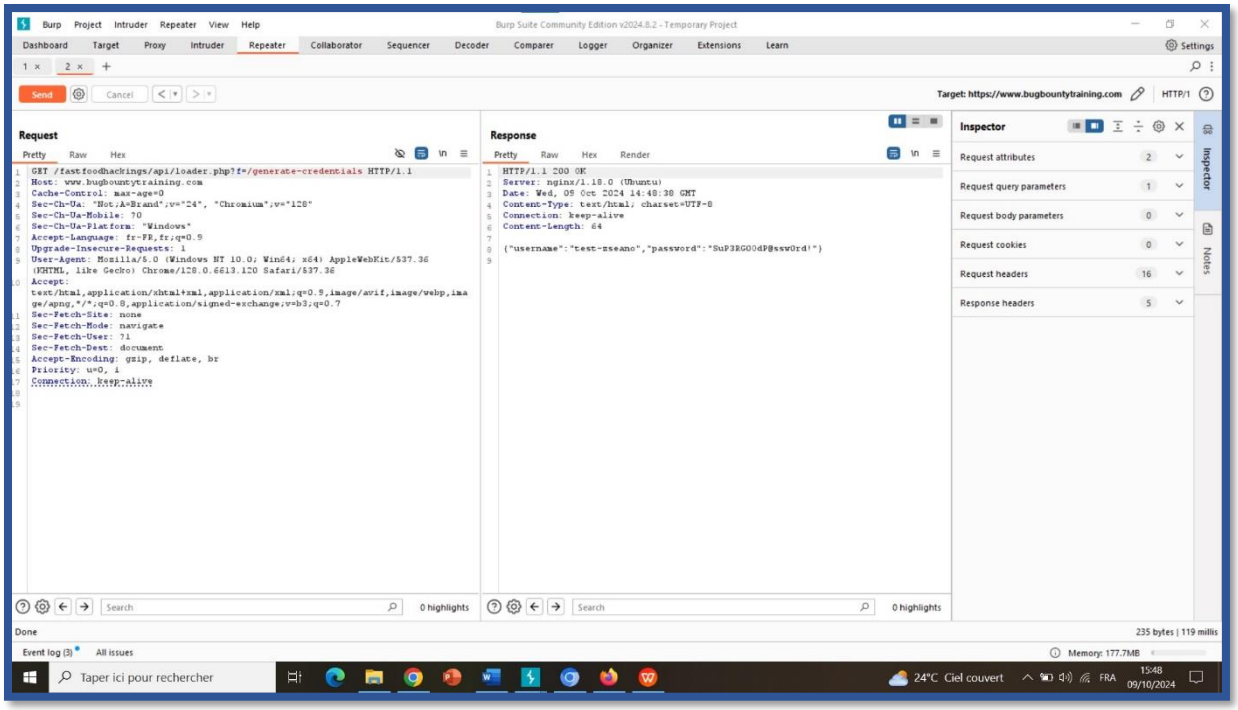

Figure 21:credentias

Et ici, vous pouvez voir notre panneau d'administration. Il peut sembler vide, mais nous avons fait un excellent travail en examinant ce paramètre et en accédant au panneau.

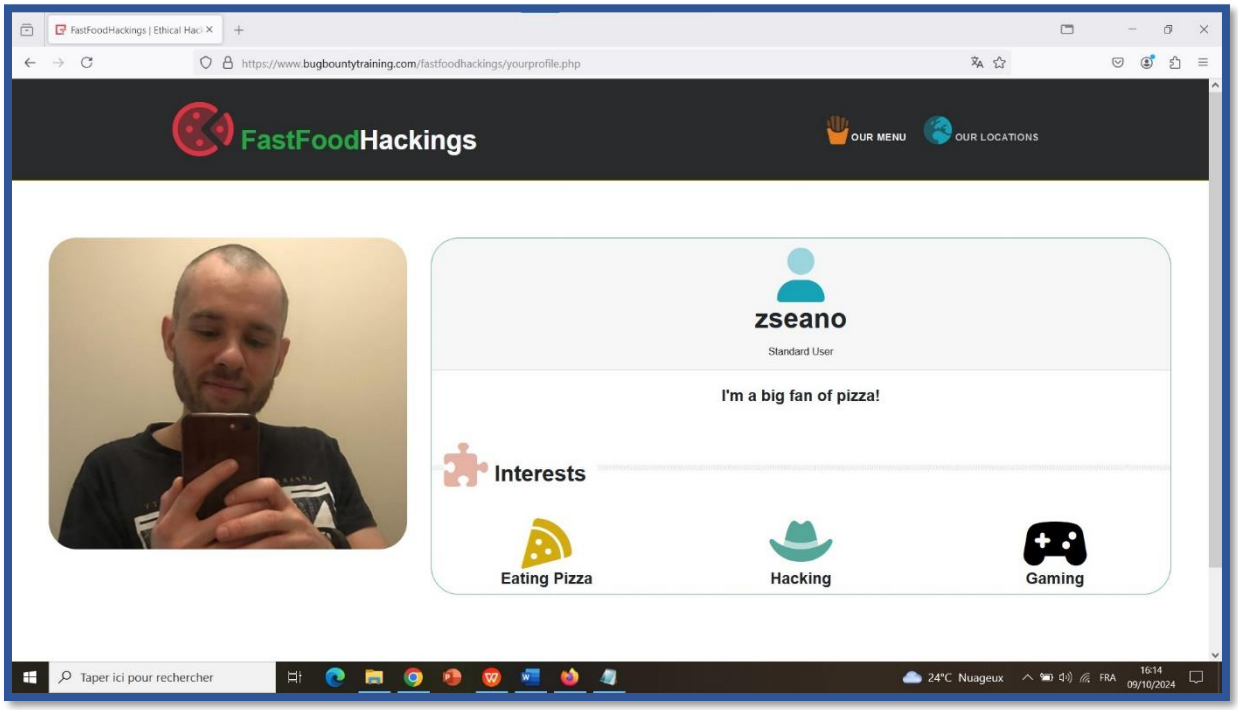

Figure 22:login page de zseno

Après avoir examiné la requête . requête inclut un id=paramètre qui pourrait valoir la peine d'être testé. Examinons-le pour voir s'il révèle une vulnérabilité de traversée de chemin.

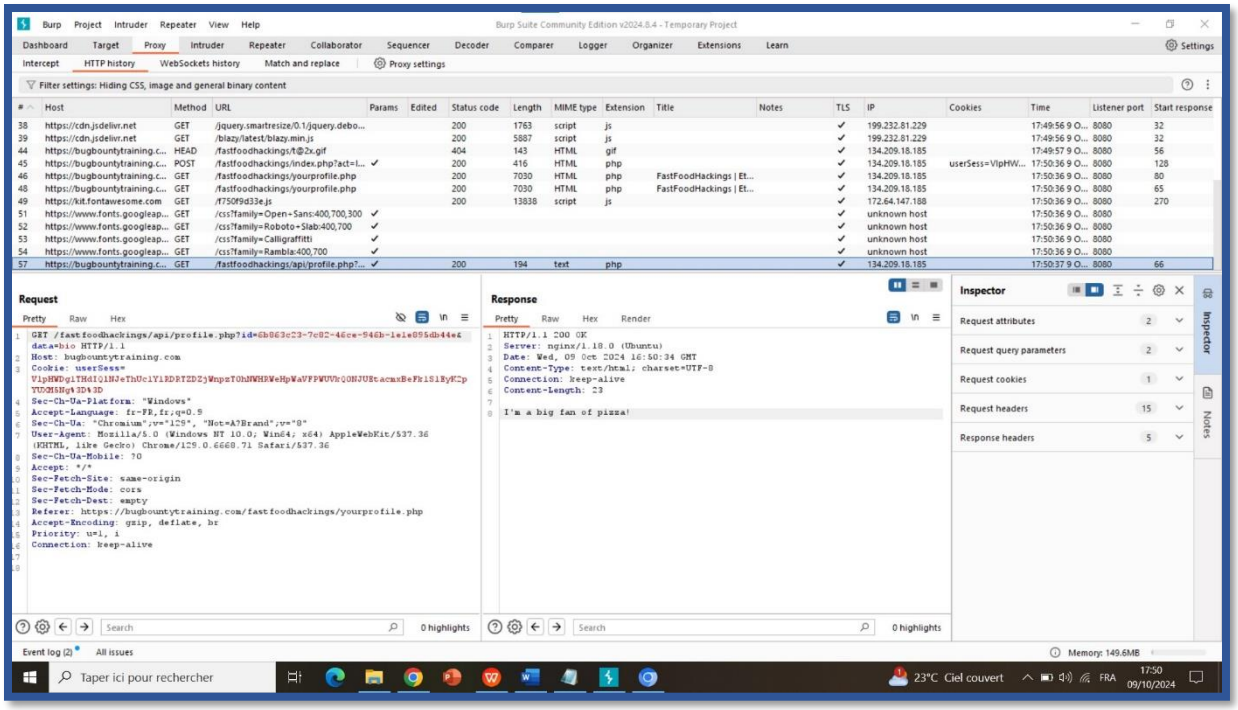

Figure 23:requête id=paramètre

IDOR:

Cependant, j'ai accidentellement changé les valeurs iden 0, 1 et d'autres petits entiers.

Alors que 0 n'a donné aucun résultat, 1 et 2 ont produit des résultats intéressants.

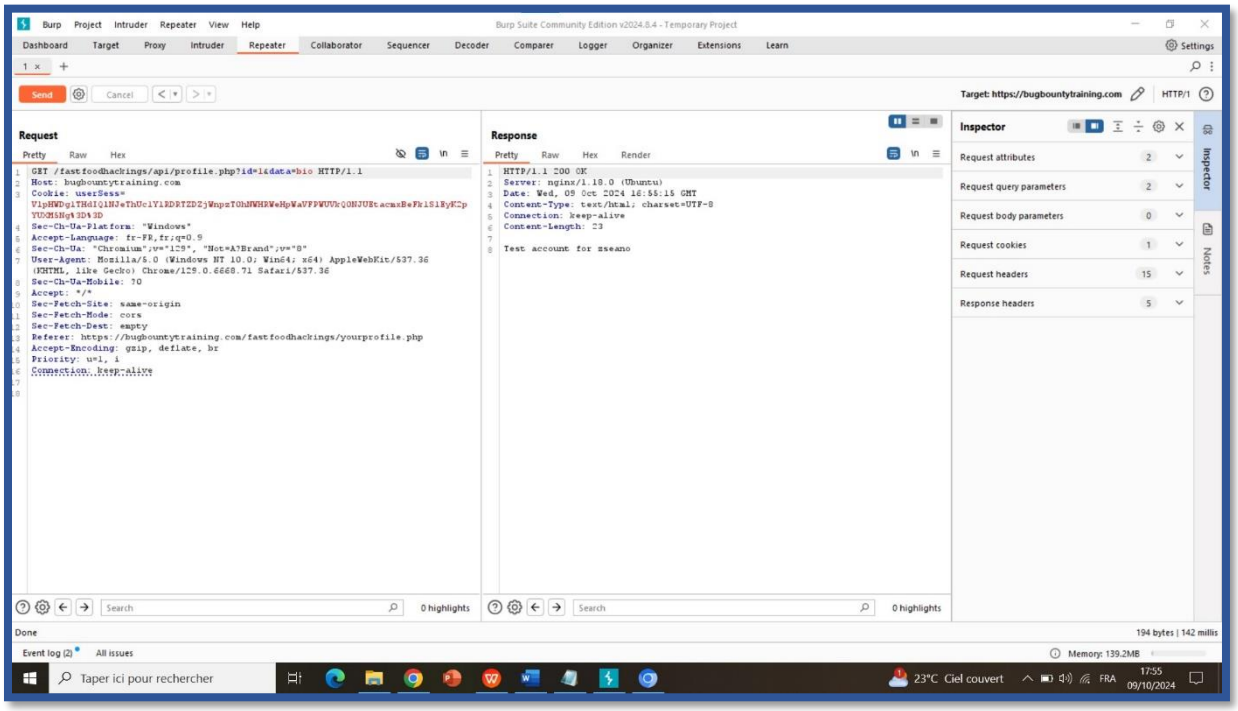

Figure 24:idor

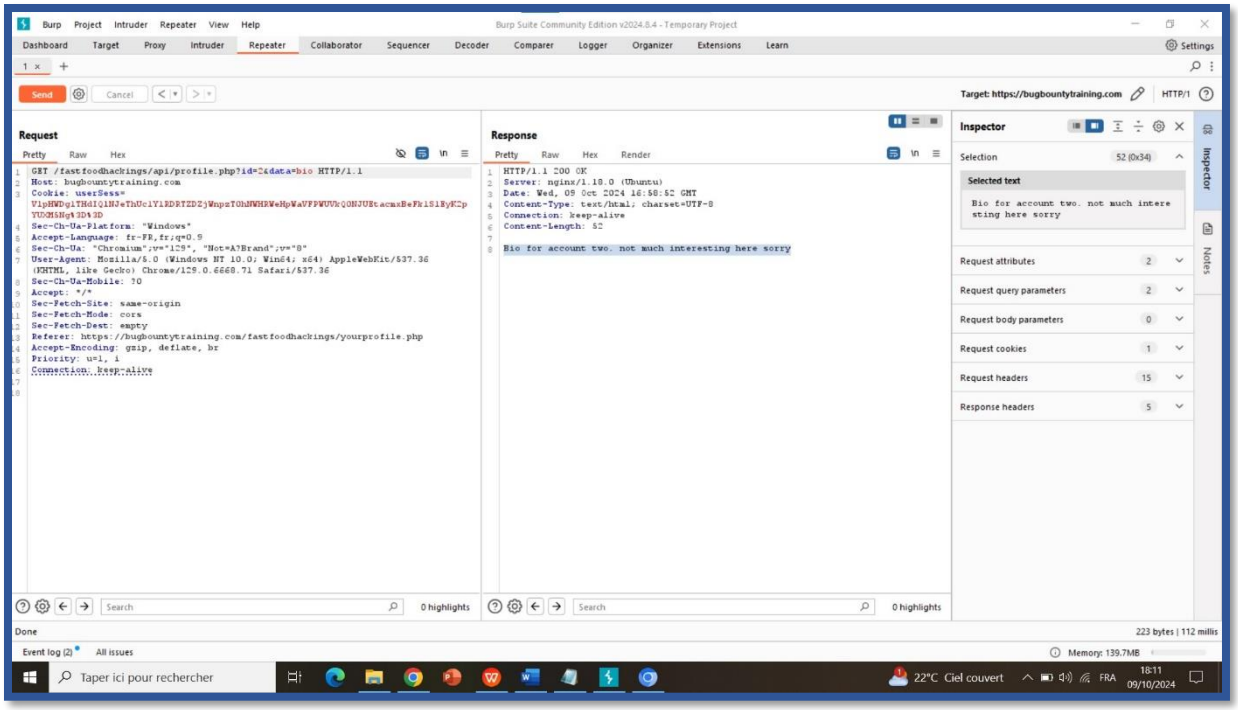

Figure 25:Insecure Direct Object Reference

Nous avons découvert une vulnérabilité IDOR (Insecure Direct Object Reference).

HTML injection :

Revenons à la page d'accueil et recherchons quelque chose d'intéressant à tester. Mon intuition suggère que nous devrions examiner le formulaire de connexion. Nous pourrions découvrir une autre vulnérabilité .

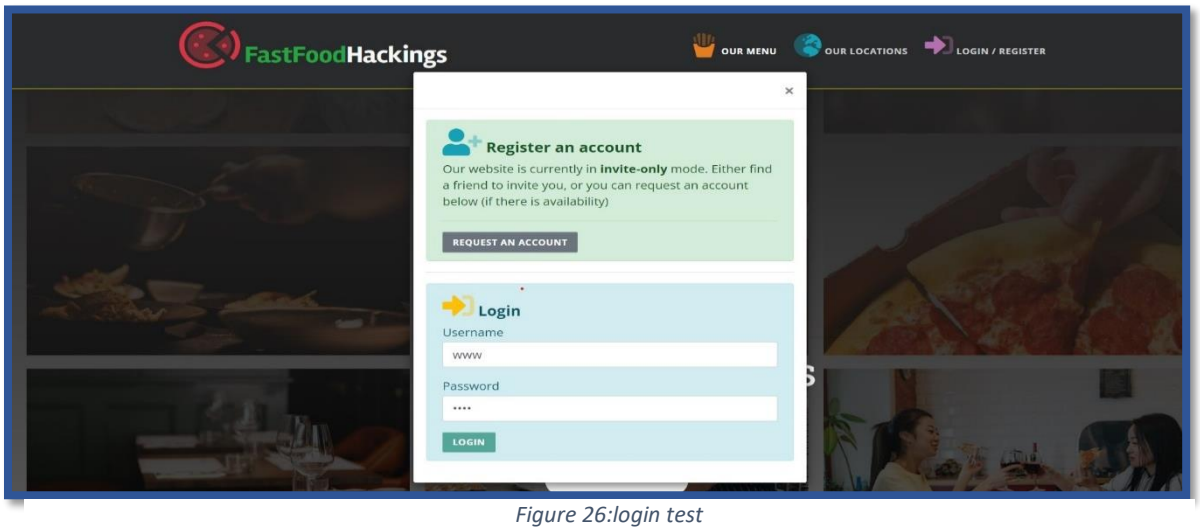

Figure 26:login test

Passons loginà autre chose, comme www, et voyons si la valeur est reflétée n'importe où sur la page. Pour vérifier, nous devons examiner le code source.

Nous l'avons trouvé commenté dans la première ligne au-dessus de la balise HTML : <!--invalid action: www-->.

Pour contourner ce problème, nous devons intégrer notre texte à la page. Nous pouvons le faire en fermant la balise de commentaire avant que notre entrée ne soit lue. Copions la balise de commentaire de fermeture (-->) et plaçons-la avant aaade fermer le commentaire.

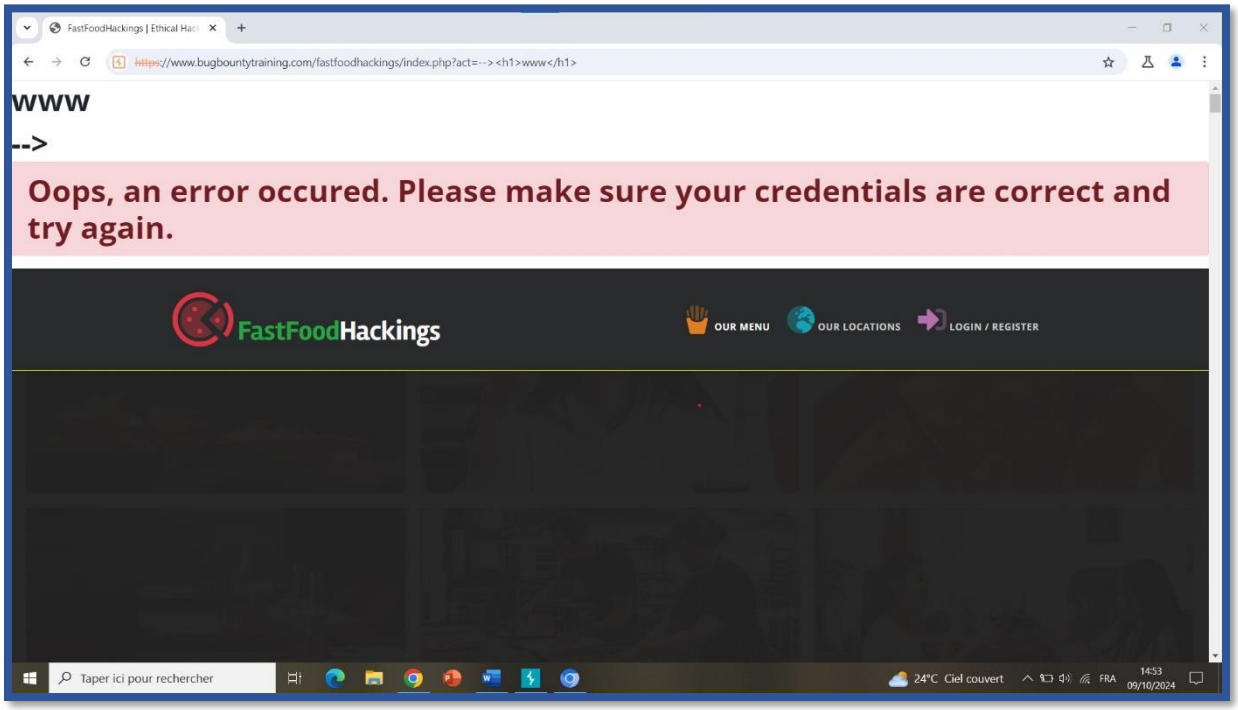

Figure 27:l'injection HTML

https://www.bugbountytraining.com/fastfoodhackings/index.php?act=--><h1>www</h1>

Heureusement , cela a fonctionné et j'ai découvert un autre bug : l'injection HTML .

Cependant, nous allons essayer d'autres méthodes et charges utiles pour contourner ce filtre. L'un de mes référentiels de charges utiles XSS préférés est le site Web **XSS.js.org** , qui contient un grand nombre de charges utiles intéressantes.

Essayons-en quelques-uns manuellement. Si cela ne fonctionne pas, nous utiliserons Burp Suite Intruder.

https://www.bugbountytraining.com/fastfoodhackings/index.php?act=--%3E%3CIMG%20SRC=x%20onerror=%22alert(String.fromCharCode(88,83,83))%22

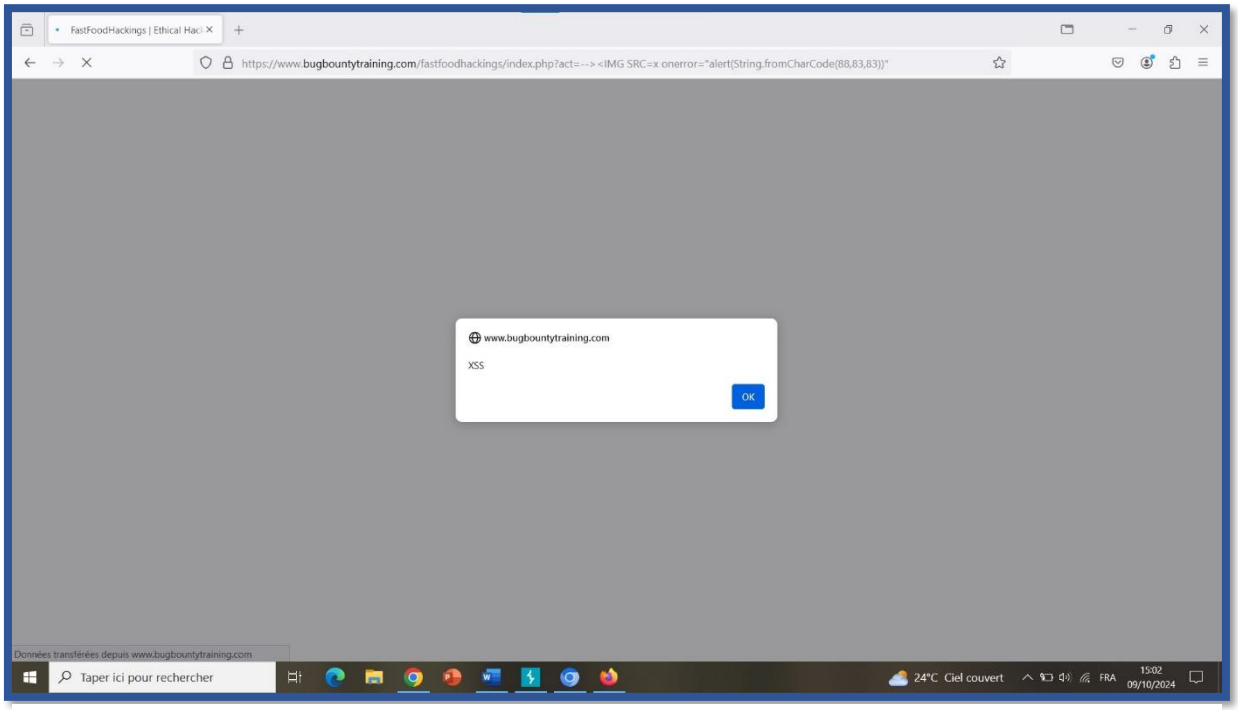

Figure 28:XSS trouvé

Après quatre tentatives infructueuses, la cinquième nous a finalement donné le résultat souhaité : **XSS trouvé !**

Ou nous pouvons simplement

< IMG SRC = x onerror = alerte ("XSS")

Broken Access Control :

Vérifions ça Code source.

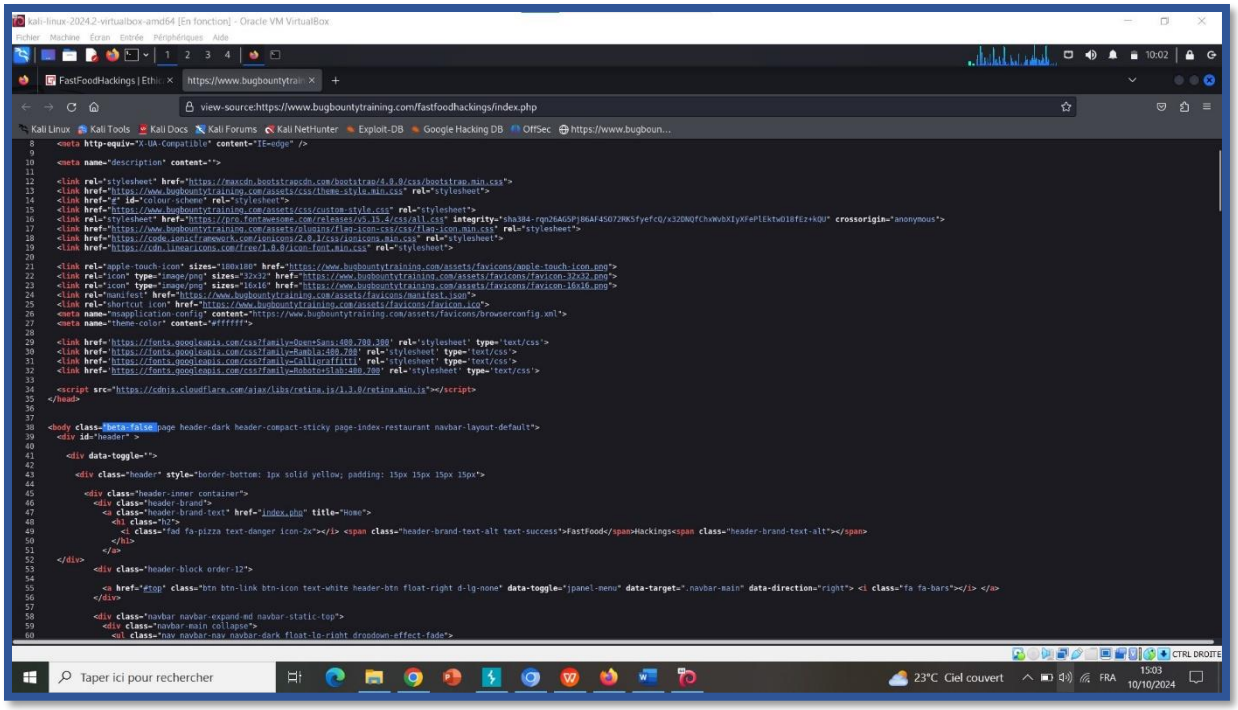

Figure 29:terme beta

Le terme BETA fait référence à une phase de test d'un produit ou d'un logiciel avant sa sortie officielle. Dans le contexte du développement de sites web, d'applications ou de logiciels, un produit "en BETA" est une version qui est encore en développement et qui est généralement testée par un groupe limité d'utilisateurs ou de développeurs pour repérer et corriger des bugs, améliorer la performance et recueillir des retours avant une sortie plus large.

Utilisation des Cookies en Version BETA :

Dans certains cas, comme sur la page que tu montres, un cookie "beta" peut être utilisé pour identifier les utilisateurs qui accèdent à la version BETA d'un site.

Ce cookie pourrait permettre de débloquer des fonctionnalités supplémentaires, d'afficher une interface légèrement différente ou d'accéder à des pages cachées que seuls les bêta-testeurs peuvent voir.

Changeons la valeur BETA et nous verrons ; Nous utiliserons l'éditeur de cookies

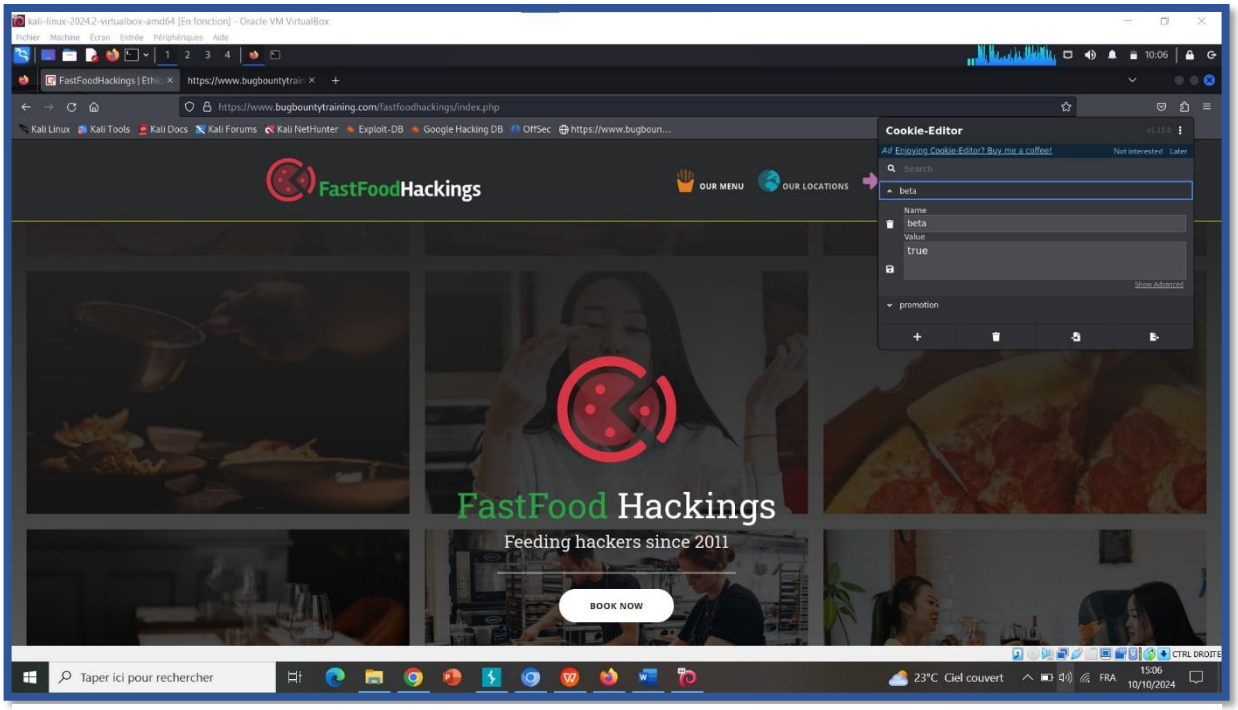

Figure 30:l'éditeur de cookies

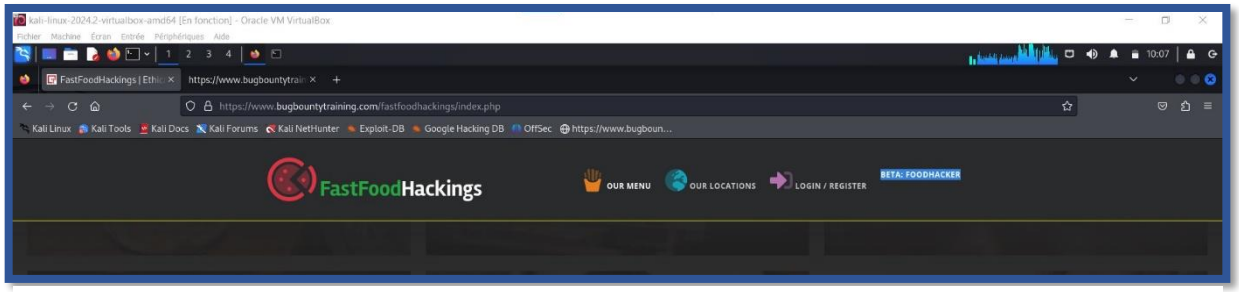

Figure 31:Bontton : Foodhacking

Clics sur la nouvelle version bêta de Bontton : Foodhacking

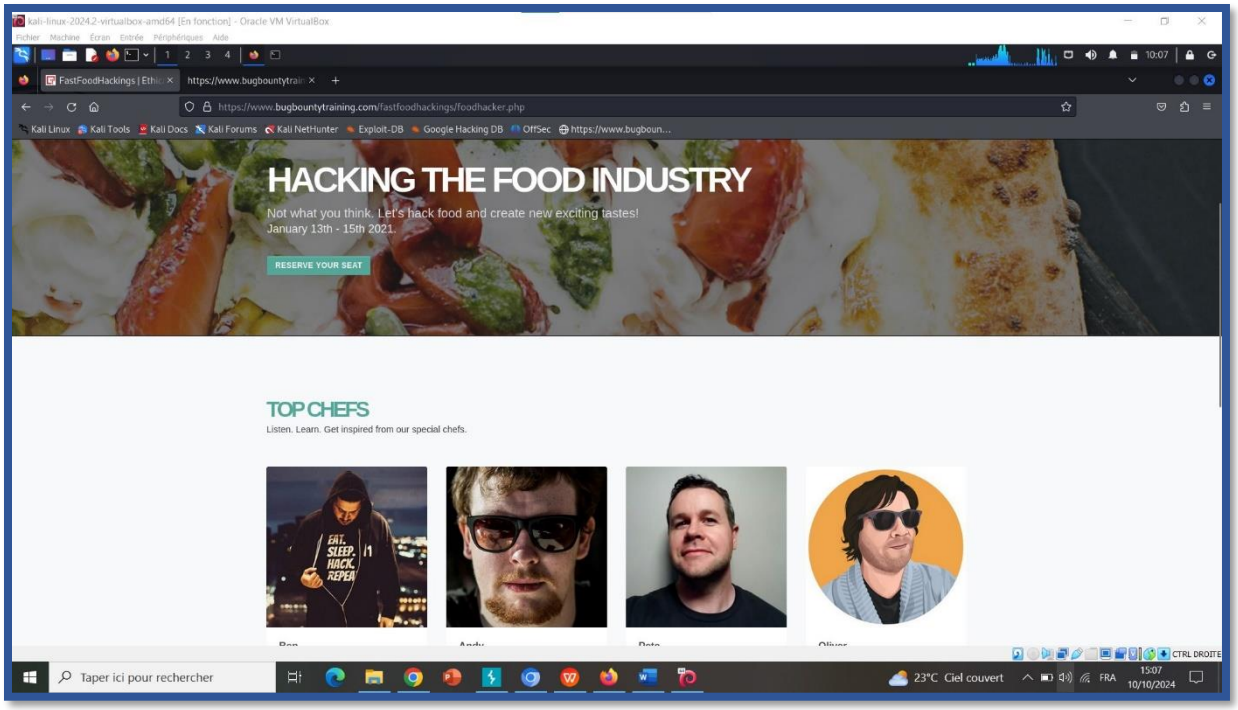

Figure 32:page sensible

Lors de l'exploration du site FastFoodHackings, une page sensible a été identifiée, accessible sans les autorisations appropriées.

Quand je consulte le site Web J'ai trouvé trois input , Vérifions-les.

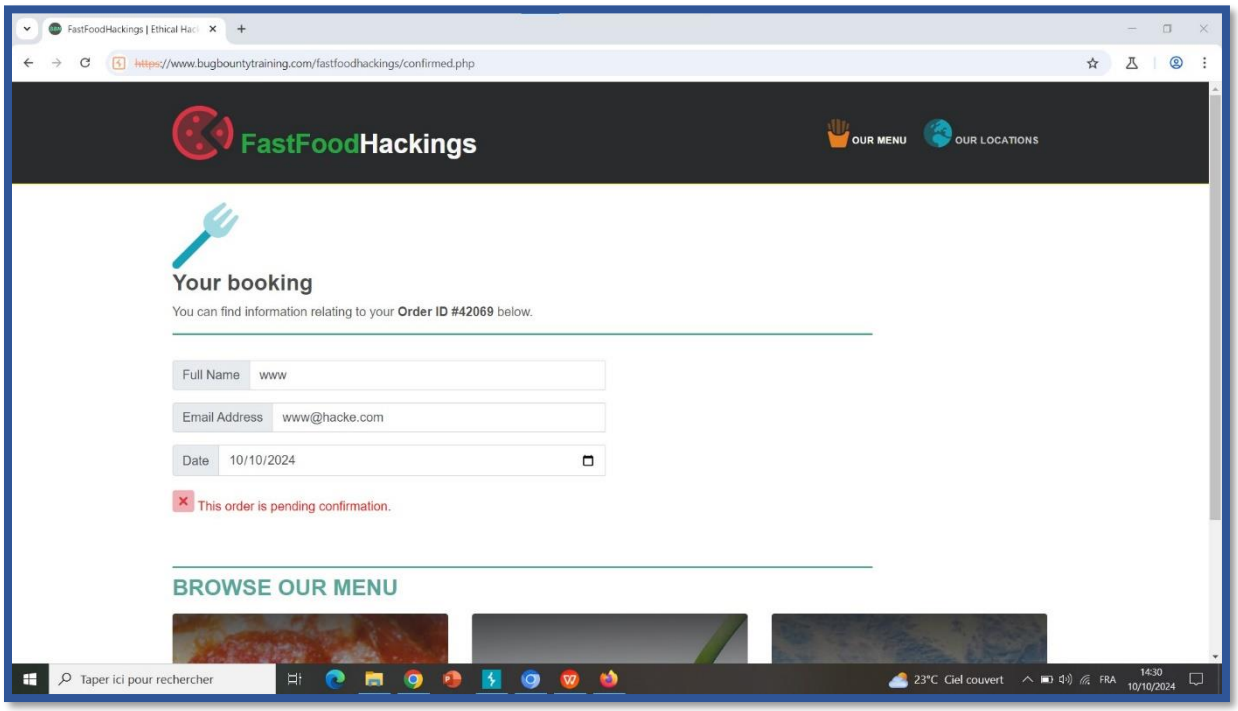

Figure 33: trois input

Envoyons cette requête à Burp Suite **Repeater** et faisons des essais. D'après mes connaissances, elle pourrait être vulnérable à une certaine forme de **contrôle d'accès brisé** .

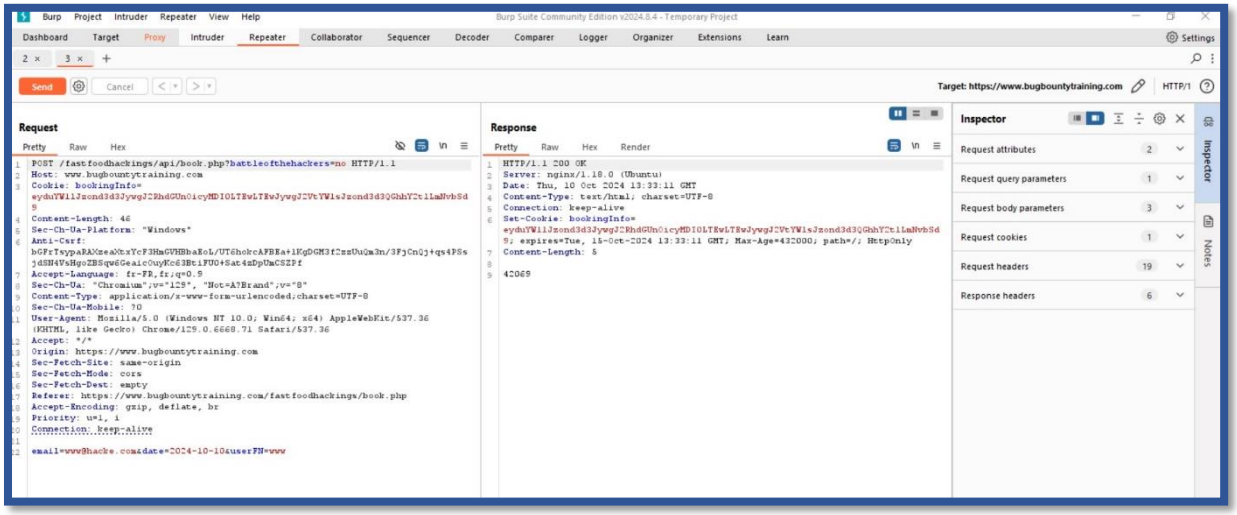

Figure 34: un id dans le reponse

Nous remarquons qu'il y a un nombre étrange , Vérifions les autre requets.

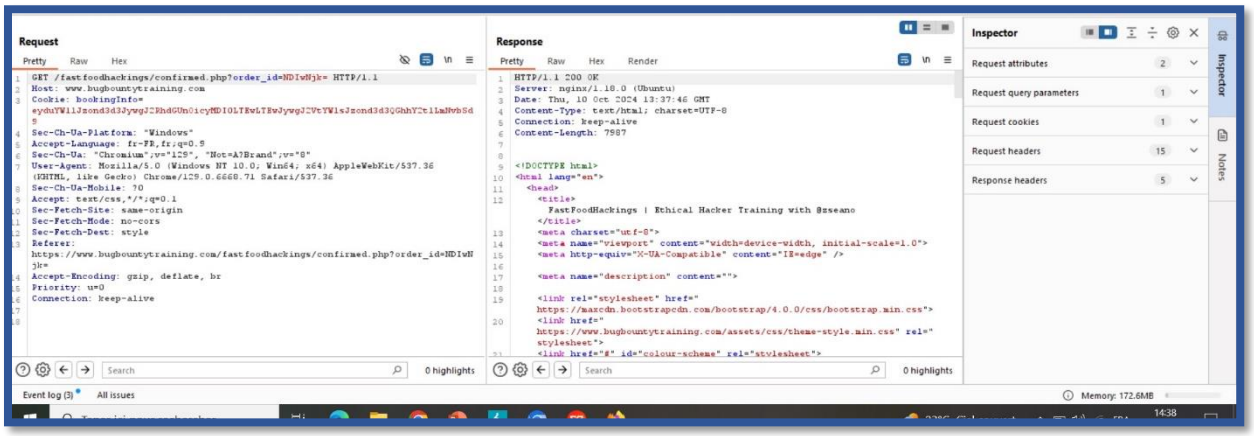

Figure 35:vérifions les autre requets

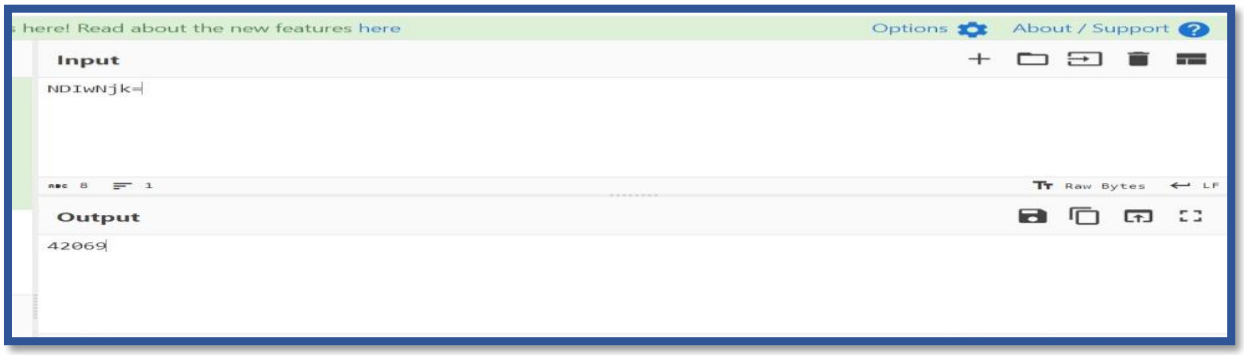

Figure 36:encoder Base64

Comme nous avons décodé la valeur de la commande et découvert qu'elle est de **42069**, expérimentons avec ce numéro pour voir si nous pouvons accéder aux commandes appartenant à d'autres utilisateurs.

Par exemple, nous pouvons le changer 42069 et observer ce qui se passe.

Mais voici l'astuce : nous devons encoder le numéro Base64avant de l'envoyer. Si nous ne le faisons pas et l'envoyons tel quel, la demande sera rejetée.

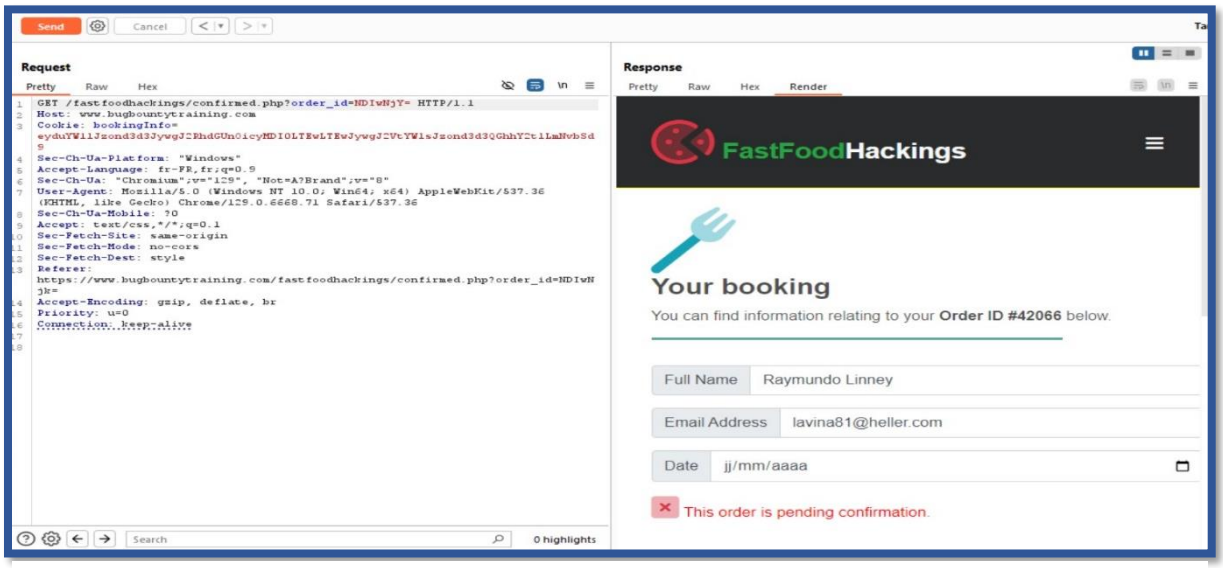

Figure 38: changer le numéro Base64

Figure 37:contrôle d'accès brisé

Comme vous pouvez le voir, quelque chose d'intéressant s'est produit. Nous avons eu accès à une autre commande soumise par une autre personne nommée Raymundo Linneyavec

l'adresse e-mail lavina81@heller.com,et une autre Adria Bossert l'adresse e-mail emory92@yahoo.com.

Cet accès n'était pas censé être disponible, illustrant un exemple de Broken Access Control.

j'ai cherché sur la nginx/1.18.0 (Ubuntu) J'ai recherché nginx/1.18.0 (Ubuntu) et je l'ai trouvé infecté par :

CVE-2020-12440 :

La vulnérabilité **CVE-2020-12440** concerne un **manque de contrôle d'accès** dans le logiciel **Dolibarr ERP/CRM**, une solution open-source utilisée par de nombreuses entreprises pour la gestion des processus métier. Cette vulnérabilité a été identifiée dans la version 11.0.3 et permet à des utilisateurs non authentifiés de lire des fichiers sensibles sur le serveur. Cela se produit en raison d'une mauvaise configuration des droits d'accès aux fichiers sur le système.

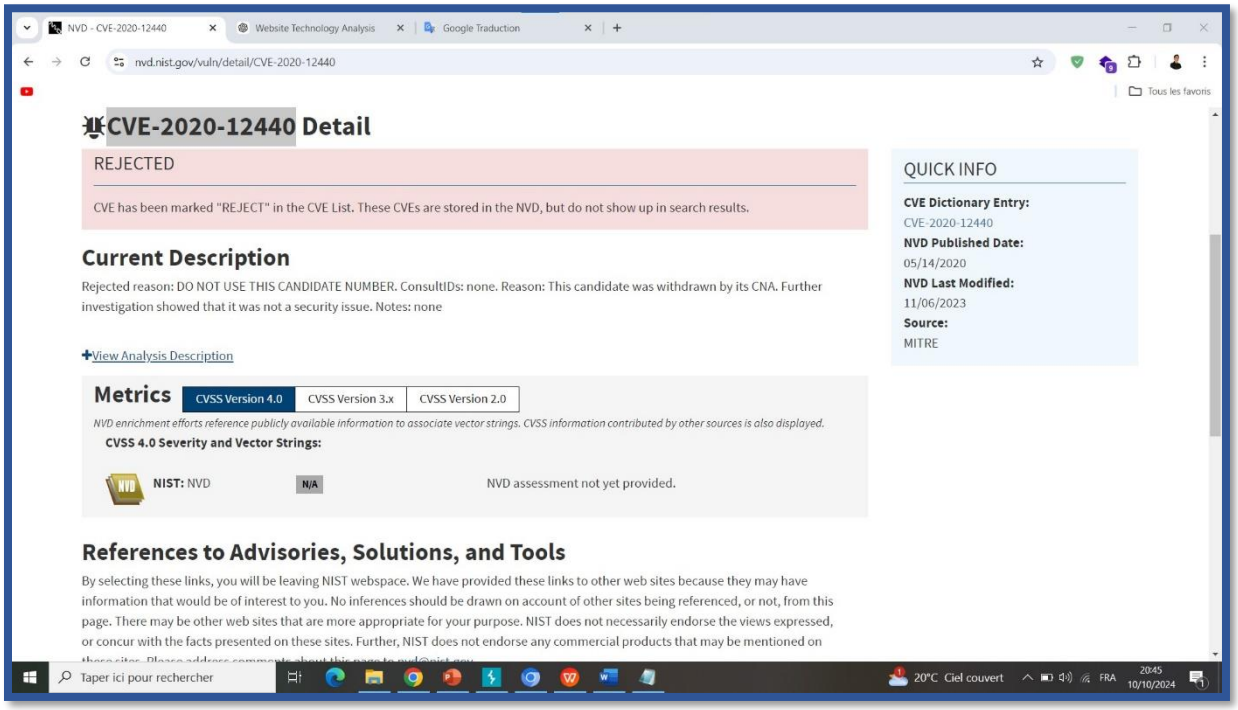

Figure 39:CVE-2020-12440

Documentation :

Clickjacking

Description : Le clickjacking est une attaque où un utilisateur est trompé pour cliquer sur des éléments invisibles ou déguisés d'une page Web, menant à des actions involontaires. Cela peut inclure des actions sensibles telles que valider une transaction ou modifier des paramètres du compte.

Impact : Les utilisateurs peuvent exécuter des actions dangereuses à leur insu, exposant l'application à des fraudes ou à des compromissions de comptes.

Recommandation : Utiliser des en-têtes HTTP comme `X-Frame-Options` avec la valeur `DENY` ou `SAMEORIGIN` pour empêcher l'application d'être chargée dans un iframe par des sites tiers.

Cross-Origin Resource Sharing

Description : Une mauvaise configuration de CORS peut permettre à des sites malveillants d'accéder aux ressources d'une application via des requêtes cross-origin non autorisées.

Impact : Cela peut mener à la fuite de données sensibles comme des informations d'utilisateur ou des jetons d'authentification.

Recommandation : Restreindre les domaines autorisés dans la politique CORS à des domaines spécifiques et légitimes, et désactiver l'accès aux données sensibles via des requêtes cross-origin.

Broken Access Control

Description : Cette vulnérabilité survient lorsque les contrôles d'accès sont mal configurés ou absents, permettant à des utilisateurs non autorisés d'accéder à des ressources restreintes.

Impact: Cela peut mener à des accès non autorisés à des fonctions d'administration, à des fichiers sensibles ou à des actions privilégiées.

Recommandation : Mettre en œuvre des contrôles d'accès robustes à tous les niveaux, vérifier les permissions utilisateur pour chaque action et tester régulièrement pour détecter les faiblesses.

IDOR (Insecure Direct Object Reference)

Description : Une vulnérabilité IDOR permet à un attaquant de manipuler les paramètres d'URL ou de requêtes pour accéder à des objets ou des ressources sans avoir les autorisations requises.

Impact : Un attaquant pourrait accéder à des comptes ou des fichiers d'autres utilisateurs sans authentification suffisante.

Recommandation : Utiliser des identifiants imprévisibles pour les ressources sensibles et vérifier les permissions à chaque demande pour s'assurer que les utilisateurs accèdent uniquement aux données qui leur appartiennent.

HTML Injection

Description : Une injection HTML permet à un attaquant d'injecter du code HTML dans une page Web, ce qui peut modifier l'apparence de la page ou introduire des éléments dangereux.

Impact : Cela peut conduire à la modification de l'interface utilisateur, à l'injection de scripts malveillants ou au vol de données utilisateur.

Recommandation : Valider et échapper toutes les entrées utilisateur avant de les rendre dans le navigateur pour éviter toute injection de contenu non autorisé.

Reflected XSS (Cross-Site Scripting)

Description : Dans une attaque XSS réfléchie, un attaquant injecte du code JavaScript malveillant qui est exécuté dans le navigateur de l'utilisateur lorsque ce dernier interagit avec des liens ou des formulaires vulnérables.

Impact : L'attaquant peut voler des cookies, usurper des sessions ou mener des attaques de phishing.

Recommandation : Utiliser une validation rigoureuse des entrées utilisateur, échapper les sorties dans les réponses HTML et appliquer une politique de sécurité des contenus (CSP) stricte.

Redirection Ouverte

Description : Cette vulnérabilité permet à un attaquant de rediriger un utilisateur vers un site malveillant en manipulant les paramètres d'URL de la redirection.

Impact: Les utilisateurs peuvent être induits en erreur et se retrouver sur des sites de phishing, mettant en péril leurs informations personnelles ou leurs identifiants.

Recommandation : Valider toutes les URL de redirection et s'assurer que seules des destinations légitimes et approuvées sont autorisées.

Fuite d'Identifiants

Description : Des informations d'identification peuvent être divulguées à la suite d'une mauvaise gestion des logs, de la configuration des erreurs ou de fuites dans le code source.

Impact : Les attaquants peuvent utiliser ces informations pour accéder à des systèmes critiques ou compromettre d'autres comptes.

Recommandation : S'assurer que les identifiants sensibles ne sont jamais exposés dans les réponses HTTP, les logs ou les messages d'erreur, et appliquer le hachage sécurisé pour le stockage des mots de passe.

CVE-2020-12440 :

Description : La CVE-2020-12440 concerne une mauvaise gestion des permissions dans Dolibarr, permettant à des utilisateurs non authentifiés d'accéder à des fichiers sensibles directement via des URL.

Impact: Cela peut conduire à l'exposition d'informations critiques comme des configurations de serveur, des bases de données ou des fichiers de sauvegarde.

Recommandation : Mettre à jour Dolibarr vers une version corrigée et vérifier les permissions sur les fichiers pour limiter l'accès uniquement aux utilisateurs autorisés.

Conclusion

L'analyse a révélé plusieurs vulnérabilités critiques dans l'application Web, certaines liées à des contrôles d'accès insuffisants, d'autres à des failles d'injection. Ces vulnérabilités pourraient être exploitées pour compromettre la confidentialité, l'intégrité et la disponibilité des données sensibles de l'application.

Recommandations générales :

- Appliquer les correctifs et les mises à jour des logiciels identifiés comme vulnérables.

- Renforcer les mécanismes d'authentification et de contrôle d'accès.

- Effectuer des tests réguliers de sécurité pour identifier d'autres failles potentielles.

- Adopter les meilleures pratiques en matière de gestion des identifiants, de validation des entrées et de configuration des serveurs.

En mettant en œuvre ces recommandations, l'application sera mieux protégée contre les attaques, réduisant ainsi les risques de compromission.